

```

        nodo↑.menor_de_segundo := menor_atrás
    end
else begin { nodo ya tiene tres hijos }
    new(ap_nuevo, interior);
    if hijo = 3 then begin
        { ap_atrás y el tercer hijo se convierten en hijos del nuevo
          nodo }
        ap_nuevo↑.primer_hijo := nodo↑.tercer_hijo;
        ap_nuevo↑.segundo_hijo := ap_atrás;
        ap_nuevo↑.tercer_hijo := nil;
        ap_nuevo↑.menor_de_segundo := menor_atrás;
        { menor_de_tercero está indefinido para ap_nuevo }
        menor := nodo↑.menor_de_tercero;
        nodo↑.tercer_hijo := nil
    end
else begin { hijo ≤ 2; pasa el tercer hijo de nodo a ap_nuevo }
    ap_nuevo↑.segundo_hijo := nodo↑.tercer_hijo;
    ap_nuevo↑.menor_de_segundo := nodo↑.menor_de_tercero;
    ap_nuevo↑.tercer_hijo := nil;
    nodo↑.tercer_hijo := nil
end;
if hijo = 2 then begin
    { ap_atrás se convierte en el primer hijo de ap_nuevo }
    ap_nuevo↑.primer_hijo := ap_atrás;
    menor := menor_atrás
end;
if hijo = 1 then begin
    { el segundo hijo de nodo pasa a ap_nuevo; ap_atrás
      se convierte en el segundo hijo de nodo }
    ap_nuevo↑.primer_hijo := nodo↑.segundo_hijo;
    menor := nodo↑.menor_de_segundo;
    nodo↑.segundo_hijo := ap_atrás;
    nodo↑.menor_de_segundo := menor_atrás
end
end
end
end; { inserta1 }
    
```

 Fig.5.19. Procedimiento *inserta1*.

Ahora es posible escribir el procedimiento INSERTA, que llama a *inserta1*. Si *inserta1* «devuelve» un nodo nuevo, entonces INSERTA debe crear una raíz nueva. El código se muestra en la figura 5.20 con la suposición de que el tipo CONJUNTO es $\uparrow$ nodo_dos_tres, esto es, un apuntador a la raíz de un árbol 2-3 cuyas hojas contienen los miembros del conjunto.

```

procedure INSERTA ( x: tipo_elemento; var S: CONJUNTO );
  var
    ap_atrás: †nodo_dos_tres; { apuntador al nuevo nodo devuelto por inserta }
    menor_atrás: real; { menor valor en el subárbol de ap_atrás }
    guardaS: CONJUNTO; { lugar para almacenar una copia temporal del
      apuntador S }
  begin
    { prueba si S está vacío o si hay un solo nodo, y debe incluirse un
      procedimiento de inserción apropiado }
    inserta(S, x, ap_atrás, menor_atrás);
    if ap_atrás <> nil then begin
      { crea la raíz nueva; sus hijos están ahora apuntados por S y ap_atrás }
      guardaS := S;
      nuevo(S);
      S.primer_hijo := guardaS;
      S.segundo_hijo := ap_atrás;
      S.menor_de_segundo := menor_atrás;
      S.tercer_hijo := nil
    end
  end; { INSERTA }

```

Fig. 5.20. INSERTA para conjuntos representados por árboles 2-3.

Realización de SUPRIME

Ahora se bosqueja una función *suprime1* que toma un apuntador al nodo *nodo* y un elemento *x*, y elimina una hoja que desciende de *nodo* que tiene el valor *x*, si es que existe †. La función *suprime1* devuelve *true* (verdadero) si después de la eliminación *nodo* tiene sólo un hijo, y devuelve *false* (falso) si *nodo* permanece con dos o tres hijos. Un esbozo del código para *suprime1* se muestra en la figura 5.21.

```

function suprime1 (nodo: †nodo_dos_tres; x: tipo_elemento ) : boolean;
  var
    sólo_uno: boolean; { para guardar el valor devuelto por una llamada a
      suprime1 }
  begin
    suprime1 := false;
    if los hijos de nodo son hojas then begin
      if x está entre esas hojas then begin
        elimina x;
        desplaza los hijos de nodo que están a la derecha de x una posición
          hacia la izquierda;
      end
    end
  end

```

† Una variante útil tomaría sólo una clave y eliminaría todo elemento con esa clave.

Fig. 5.21. Procedimiento recursivo de supresión.

El código detallado de la función *suprime1* se deja como ejercicio. Otro ejercicio es escribir un procedimiento SUPRIME (*S*, *x*) que pruebe los casos especiales en

que el conjunto S consta de una sola hoja o está vacío, y en otro caso llame a *suprimel* (S, x); si *suprimel* devuelve *true*, el procedimiento elimina la raíz (el nodo al que apunta S) y hace que S apunte al hijo que quedó solo.

5.5 Conjuntos con las operaciones COMBINA y ENCUENTRA

En ciertos problemas, se empieza con una colección de objetos, cada uno de ellos contenido en un conjunto; después se combinan los conjuntos en algún orden dado, y de vez en cuando se pregunta en qué conjunto se encuentra algún elemento en particular. Estos problemas pueden resolverse por medio de las operaciones COMBINA y ENCUENTRA. La operación COMBINA(A, B, C) hace C igual a la unión de los conjuntos A y B , bajo el supuesto de que A y B son disjuntos (no tienen miembros en común); COMBINA está indefinida si A y B no son disjuntos. ENCUENTRA(x) es una función que devuelve el conjunto del cual x es un miembro; en caso de que x esté en dos o más conjuntos, o en ninguno, ENCUENTRA no está definida.

Ejemplo 5.6. Una *relación de equivalencia* es una relación reflexiva, simétrica y transitiva. Esto es, si $\equiv$ es una relación de equivalencia en el conjunto S , para cualesquiera miembros a, b y c de S (no necesariamente distintos), se cumplen las siguientes propiedades:

1. $a \equiv a$ (*reflexividad*).
2. Si $a \equiv b$, entonces $b \equiv a$ (*simetría*).
3. Si $a \equiv b$ y $b \equiv c$, entonces $a \equiv c$ (*transitividad*).

La relación «igual a» ($=$) es la relación de equivalencia ejemplar en cualquier conjunto S . Para a, b y c de S , se tiene 1) $a = a$, 2) si $a = b$, entonces $b = a$, y 3) si $a = b$ y $b = c$, entonces $a = c$. Existen muchas otras relaciones de equivalencia, sin embargo, y pronto se verán varios ejemplos adicionales.

En general, siempre que se divide una colección de objetos en grupos disjuntos, la relación $a = b$ es de equivalencia si, y sólo si, a y b están en el mismo grupo. «Igual a» es el caso especial donde todo elemento está en grupo por sí solo.

Más formalmente, si un conjunto S tiene una relación de equivalencia definida en él, el conjunto S puede dividirse en subconjuntos disjuntos $S_1, S_2, \dots$, llamados *clases de equivalencia*, cuya unión es S . Cada subconjunto S_i consta de miembros equivalentes de S . Esto es, $a = b$ para todo a y b en S_i , y $a \neq b$ si a y b están en subconjuntos diferentes. Por ejemplo, la relación congruencia módulo n $\dagger$ es una relación de equivalencia en el conjunto de los enteros. Para comprobar que esto es así, obsérvese que $a - a = 0$, que es un múltiplo de n (reflexividad); si $a - b = dn$, entonces $b - a = (-d)n$ (simetría), y si $a - b = dn$ y $b - c = en$, entonces $a - c = (d + e)n$ (transitividad). En el caso de la congruencia módulo n existen n clases de equiva-

$\dagger$ Se dice que a es congruente con b módulo n si a y b tienen los mismos residuos cuando se dividen entre n , o dicho de otra forma, $a - b$ es múltiplo de n .

lencia, las cuales son el conjunto de los enteros congruentes con 0, el conjunto de los enteros congruentes con 1, ..., el conjunto de los enteros congruentes con $n - 1$.

El *problema de equivalencia* puede formularse de la siguiente manera. Se dan un conjunto S y una secuencia de proposiciones de la forma « a es equivalente a b ». Hay que procesar las proposiciones en orden, de manera que en cualquier momento pueda determinarse a qué clase de equivalencia pertenece un elemento dado. Por ejemplo, supóngase que $S = \{1, 2, \dots, 7\}$ y se tiene la secuencia de proposiciones

$$1=2 \quad 5=6 \quad 3=4 \quad 1=4$$

para procesar. Es necesario construir la siguiente secuencia de clases de equivalencia, suponiendo que inicialmente cada elemento de S está en una clase de equivalencia propia.

$$1=2 \quad \{1,2\} \quad \{3\} \quad \{4\} \quad \{5\} \quad \{6\} \quad \{7\}$$

$$5=6 \quad \{1,2\} \quad \{3\} \quad \{4\} \quad \{5,6\} \quad \{7\}$$

$$3=4 \quad \{1,2\} \quad \{3,4\} \quad \{5,6\} \quad \{7\}$$

$$1=4 \quad \{1,2,3,4\} \quad \{5,6\} \quad \{7\}$$

Se puede «resolver» el problema de equivalencia empezando con cada elemento de un conjunto determinado. Al procesar la proposición $a=b$, se ENCUENTRAN las clases de equivalencia de a y b , y después se COMBINAN. En cualquier momento se puede usar ENCUENTRA para conocer la clase de equivalencia actual de cualquier elemento.

El problema de equivalencia surge en varias áreas de las ciencias de la computación. Por ejemplo, una forma ocurre cuando un compilador de Fortran tiene que procesar «declaraciones de equivalencia» como

$$\text{EQUIVALENCE (A(1), B(1, 2), C(3)), (A(2), D, E), (F, G)}$$

Otro ejemplo, presentado en el capítulo 6, usa soluciones al problema de equivalencia para ayudar a encontrar árboles abarcadores de costo mínimo. □

Una realización simple de CONJUNTO_CE

Ahora se presenta una versión simplificada del TDA COMBINA_ENCuentra, al definir un TDA llamado CONJUNTO_CE, que consiste en un conjunto de subconjuntos llamados *componentes*, junto con las siguientes operaciones:

1. COMBINA(A, B), que toma la unión de los componentes A y B y al resultado lo llama A o B , arbitrariamente.

2. **ENCUENTRA**(x), que es una función que devuelve el nombre del componente del cual x es un miembro.
3. **INICIAL**(A, x), que crea un componente llamado A que contiene sólo el elemento x .

Para hacer una realización razonable de **CONJUNTO_CE**, se deben restringir los tipos o reconocer que **CONJUNTO_CE** en realidad tiene otros dos tipos como «parámetros»: el tipo de los nombres de los conjuntos y el tipo de los miembros de esos conjuntos. En muchas aplicaciones se pueden usar enteros como nombres de conjuntos. Si n es el número de elementos, también se pueden usar enteros en el intervalo $[1..n]$ para los miembros de los componentes. Para la implantación en cuestión, es importante que el tipo de los miembros de los conjuntos sea del tipo subintervalo, porque se desea indizar en un arreglo definido en él. El tipo de los nombres de los conjuntos no es importante, pues es del tipo de los elementos del arreglo, no de sus índices. Obsérvese, sin embargo, que si se desea que el tipo de los miembros sea distinto a un subintervalo, se puede crear una correspondencia con una tabla de dispersión, por ejemplo, que los asigne a enteros únicos de un subintervalo. Sólo es necesario conocer por adelantado el número total de elementos.

La aplicación que se está considerando es declarar

```
const
    n = {número de elementos};
type
    CONJUNTO_CE = array[1..n] of integer;
```

como un caso especial del tipo más general

array[subintervalo de miembros] of (tipo de nombres de los conjuntos);

Supóngase que se declaran *componentes* del tipo **CONJUNTO_CE** con la intención de que *componentes*[x] contenga el nombre del conjunto en el cual se encuentra x . Entonces, las tres operaciones para **CONJUNTO_CE** son fáciles de escribir. Por ejemplo, la operación **COMBINA** se muestra en la figura 5.22. **INICIAL**(A, x) simplemente hace que *componentes*[x] sea igual a A , y **ENCUENTRA**(x) devuelve *componentes*[x].

```
procedure COMBINA ( A, B: integer; var C: CONJUNTO_CE );
var
    x: 1..n;
begin
    for x := 1 to n do
        if C[x] = B then
            C[x] := A
    end; { COMBINA }
```

Fig. 5.22. El procedimiento **COMBINA**.

El rendimiento en tiempo de esta implantación de CONJUNTO_CE es fácil de analizar. Cada ejecución del procedimiento COMBINA lleva un tiempo $O(n)$. Por otro lado, las implantaciones obvias de INICIAL(A, x) y ENCUENTRA(x) tienen tiempos de ejecución constantes.

Realización más rápida de CONJUNTO_CE

Al utilizar el algoritmo de la figura 5.22, una secuencia de $n-1$ operaciones COMBINA llevará un tiempo $O(n^2)$ †. Una forma de acelerar la operación COMBINA es al encadenar todos los miembros de un componente en una lista. Entonces, en vez de leer todos los miembros cuando se combina el componente B en A , basta recorrer la lista de los miembros de B . Esta organización aprovecha mejor el tiempo en el caso promedio. Sin embargo, podría suceder que la i -ésima combinación fuera de la forma COMBINA(A, B), donde A sería un componente de tamaño uno y B sería un componente de tamaño i , y que el resultado se llamara B . Esta operación COMBINA requeriría $O(i)$ pasos, y una secuencia de $n-1$ instrucciones COMBINA llevaría un tiempo del orden de $\sum_{i=1}^{n-1} i = n(n-1)/2$.

Una forma de evitar esta situación del peor caso es cuidar el tamaño de cada componente y combinar siempre el más pequeño dentro del más grande ††. Así, cada vez que un miembro se combina con un componente más grande, se encuentra a sí mismo en un componente al menos dos veces más grande. De esta forma, si existen n componentes inicialmente, cada uno con un miembro, ninguno de los n miembros puede tener su componente cambiado más de $1 + \log n$ veces. Como el tiempo consumido por esta nueva versión de COMBINA es proporcional al número de miembros cuyos nombres de componentes se cambian, y el número total de tales cambios puede ser hasta $n(1 + \log n)$, el trabajo $O(n \log n)$ basta para todas las combinaciones.

Ahora, considérese la estructura de datos necesaria para esta realización. Primero se necesita una correspondencia de los nombres de conjuntos con los registros que consisten en

1. un contador que da el número de miembros del conjunto y
2. el índice en el arreglo del primer elemento de ese conjunto.

También se necesita otro arreglo de registros, indizados de acuerdo con los miembros, para indicar

1. el conjunto del cual cada elemento es miembro y
2. el siguiente elemento del arreglo en la lista de ese conjunto.

† Obsérvese que $n-1$ es el mayor número de combinaciones que pueden hacerse antes de que todos los elementos estén en un solo conjunto.

†† Obsérvese que la capacidad para llamar al componente resultante de acuerdo con el nombre de cualquiera de sus elementos es importante aquí, aunque en la realización más sencilla se escoge siempre el nombre del primer argumento.

Se emplea 0 como equivalente a NIL, la marca de fin de lista. En un lenguaje que se preste para este tipo de construcciones, sería preferible el uso de apuntadores en este arreglo, pero Pascal no permite apuntadores en los arreglos.

En el caso especial donde los nombres de los conjuntos, al igual que los miembros, se escogen del subintervalo $1..n$, es posible usar un arreglo para la correspondencia descrita antes. Esto es, se define

```

type
  tipo_nombre = 1..n;
  tipo_elemento = 1..n;
  CONJUNTO_CE = record
    encabezamientos_conjuntos : array[1..n] of record
      {encabezamientos para las listas de conjuntos}
      contador: 0..n;
      primer_elemento: 0..n
    end;
    nombres: array[1..n] of record
      {tabla que da los conjuntos que contienen a cada miembro}
      nombre_conjunto: tipo_nombre;
      siguiente_elemento: 0..n
    end
  end
end;
```

Los procedimientos INICIAL, COMBINA y ENCUENTRA se muestran en la figura 5.23.

```

procedure INICIAL ( A: tipo_nombre; x: tipo_elemento; var C: CONJUNTO_CE );
{ asigna como valor inicial de A un conjunto que sólo contiene a x }
begin
  C.nombres[x].nombre_conjunto := A;
  C.nombres[x].siguiente_elemento := 0;
  { apuntador nulo al final de la lista de los miembros de A }
  C.encabezamientos_conjuntos[A].cuenta := 1;
  C.encabezamientos_conjuntos[A].primer_elemento := x
end; { INICIAL }
```

```

procedure COMBINA ( A, B: tipo_nombre; var C: CONJUNTO_CE );
{ combina A y B y llama al resultado A o B, arbitrariamente }
var
  i: 0..n; { se usa para encontrar el fin de la lista más pequeña }
begin
  if C.encabezamientos_conjuntos[A].cuenta >
    C.encabezamientos_conjuntos[B].cuenta then begin
    { A es el conjunto más grande; combina B dentro de A }
    { encuentra el final de B, cambiando los nombres de los conjuntos
      por A conforme se avanza }
    i := C.encabezamientos_conjuntos[B].primer_elemento;
```

repeat

C.nombres[i].nombre_conjunto := A;

i := C.nombres[i].siguiente_elemento

until *C.nombres[i].siguiente_elemento = 0;*

{ añade la lista *A* al final de la *B* y llama *A* al resultado }

{ ahora *i* es el índice del último miembro de *B* }

C.nombres[i].nombre_conjunto := A;

C.nombres[i].siguiente_elemento :=

C.encabezamientos_conjuntos[A].primer_elemento;

C.encabezamientos_conjuntos[A].primer_elemento :=

C.encabezamientos_conjuntos[B].primer_elemento;

C.encabezamientos_conjuntos[A].cuenta :=

C.encabezamientos_conjuntos[A].cuenta +

C.encabezamientos_conjuntos[B].cuenta;

C.encabezamientos_conjuntos[B].primer_elemento := 0

C.encabezamientos_conjuntos[B].primer_elemento := 0

{ los dos pasos anteriores no son realmente necesarios, pues el conjunto *B* ya no existe }

end

else { *B* es al menos tan grande como *A* }

{ código similar al del caso anterior, pero con *A* y *B* intercambiados }

end; { COMBINA }

function ENCUENTRA (*x: 1..n; var C: CONJUNTO_CE*);

{ devuelve el nombre de aquel conjunto que tiene a *x* como miembro }

begin

return (*C.nombres[x].nombre_conjunto*)

end; { ENCUENTRA }

Fig. 5.23. Las operaciones de un CONJUNTO_CE.

La figura 5.24 muestra un ejemplo de la estructura de datos empleada en la figura 5.23, donde el conjunto 1 es {1, 3, 4}, el conjunto 2 es {2}, y el conjunto 5 es {5, 6}.

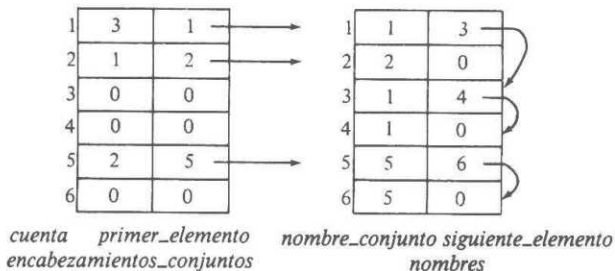


Fig. 5.24. Ejemplo de la estructura de datos CONJUNTO_CE.

Realización de CONJUNTO_CE con árboles

Otro enfoque completamente distinto para la realización de CONJUNTO_C usa árboles con apuntadores a los padres. Se describirá de manera informal este enfoque. La idea básica es que los nodos de los árboles se correspondan con los miembros del conjunto, con un arreglo u otra realización de una correspondencia que vaya de los miembros del conjunto a sus nodos. A excepción de la raíz del árbol, cada nodo tiene un apuntador a su padre. Las raíces tienen el nombre del conjunto, además de un elemento. Una correspondencia de los nombres del conjunto con las raíces permite el acceso a cualquier conjunto dado al hacer las combinaciones.

La figura 5.25 muestra los conjuntos $A = \{1, 2, 3, 4\}$, $B = \{5, 6\}$ y $C = \{7\}$ representados de esta forma. Se supone que los rectángulos son parte del nodo raíz, no nodos independientes.

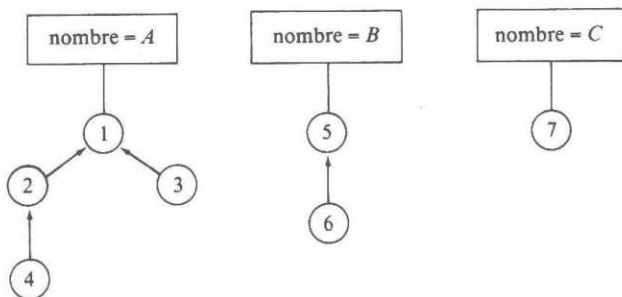
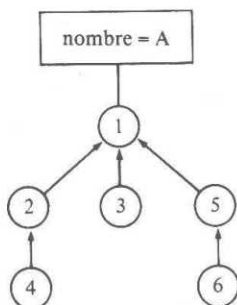


Fig. 5.25. CONJUNTO_CE representado por una colección de árboles.

Para encontrar el conjunto que contiene un elemento x , primero se consulta una correspondencia (por ejemplo, un arreglo), que no se muestra en la figura 5.25, para obtener un apuntador al nodo para x . Después, se sigue el camino de ese nodo a la raíz de su árbol para leer el nombre de ese conjunto.

La operación de combinación básica consiste en hacer que la raíz de un árbol sea el hijo de la raíz del otro. Por ejemplo, se podrían combinar A y B de la figura 5.25 y llamarle A al resultado, haciendo que el nodo 5 sea hijo del nodo 1. El resultado se muestra en la figura 5.26. Sin embargo, la combinación indiscriminada podría producir un árbol de n nodos que fuera una sola cadena. Entonces, hacer la operación ENCUENTRA en cada uno de esos nodos llevaría un tiempo $O(n^2)$. Obsérvese que aunque una combinación se puede hacer en $O(1)$ pasos, el costo de un número razonable de procedimientos ENCUENTRA dominaría en el costo total, y este enfoque no es necesariamente mejor que uno más simple para ejecutar n procedimientos combina y n encuentra.

Sin embargo, una mejora sencilla garantiza que si n es el número de elementos, ningún ENCUENTRA necesitará más de $O(\log n)$ pasos. Sólo se conserva un contador del número de elementos del conjunto en cada raíz, y al intentar combinar dos

Fig. 5.26. Combinación de *B* dentro de *A*.

conjuntos, se hace que la raíz del árbol más pequeño sea un hijo de la raíz del más grande. Así, cada vez que un nodo pasa a un árbol nuevo, suceden dos cosas: la distancia del nodo a su raíz se incrementa en 1, y el nodo estará en un conjunto con por lo menos el doble de elementos que antes. Así, si n es el número total de elementos, ningún nodo puede moverse más de $\log n$ veces; la distancia a su raíz nunca puede exceder de $\log n$. Se concluye que cada *ENCUENTRA* requiere un máximo de tiempo $O(\log n)$.

Compresión de caminos

Otra idea que puede acelerar esta realización de *CONJUNTO_CE* es una *compresión de caminos*. Durante un procedimiento *ENCUENTRA*, al seguir un camino desde algún nodo hasta la raíz, se hace que cada nodo encontrado en el camino sea un hijo de la raíz. La forma más fácil de realizar esto es en dos pasos. Primero se encuentra la raíz y después se recorre de nuevo el mismo camino, haciendo que cada nodo sea hijo de la raíz.

Ejemplo 5.7. La figura 5.27(a) muestra un árbol antes de ejecutar una operación *ENCUENTRA* en el nodo del elemento 7, y la figura 5.27(b) muestra el resultado después de quedar 5 y 7 como hijos de la raíz. Los nodos 1 y 2 del camino no se mueven porque 1 es la raíz y 2 ya es hijo de la raíz. □

La compresión de caminos no afecta al costo de los procedimientos *COMBINA*; cada uno de ellos continúa consumiendo una cantidad constante de tiempo. Sin embargo, existe un ligero incremento en la velocidad de *ENCUENTRA*, ya que la compresión de caminos tiende a acortar un número grande de ellos, desde varios nodos hasta la raíz, con relativamente poco esfuerzo.

Desafortunadamente, es muy difícil analizar el costo promedio de *ENCUENTRA* cuando se usa la compresión de caminos. De manera que si no es necesario que los árboles más cortos se combinen dentro de los más grandes, no se requerirá

más tiempo que $O(n \log n)$ para hacer n procedimientos ENCUENTRA. Por supuesto, el primero de ellos puede llevar un tiempo $O(n)$ por sí mismo para un árbol que conste de una cadena. Pero la compresión puede cambiar muy rápido un árbol, y con independencia del orden de aplicación de ENCUENTRA a los elementos de cualquier árbol, no se necesitará un tiempo mayor que $O(n)$ para n procedimientos ENCUENTRA. Sin embargo, existen secuencias de las instrucciones COMBINA y ENCUENTRA que requiere un tiempo $\Omega(n \log n)$.

El algoritmo que usa compresión de caminos y combina los árboles más pequeños dentro de los más grandes es asintóticamente el método más eficiente conocido para aplicar CONJUNTO_CE. En particular, n procedimientos ENCUENTRA no requieren un tiempo mayor que $O(n\alpha(n))$, donde $\alpha(n)$ es una función no constante, pero que crece mucho más lentamente que $\log n$. Se definirá $\alpha(n)$ enseguida, pero el análisis que da lugar a esta cota está fuera del alcance de este libro.

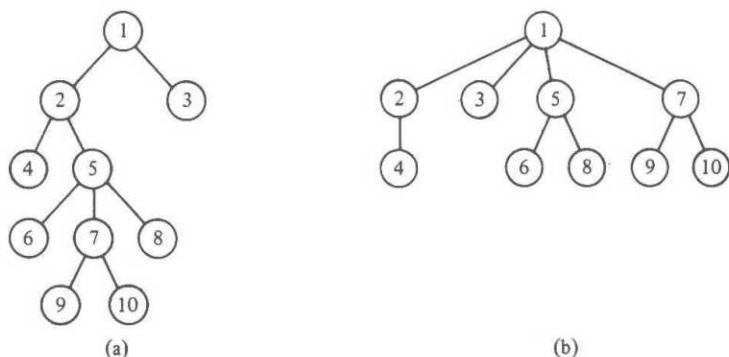


Fig. 5.27. Ejemplo de compresión de caminos.

Función $\alpha(n)$

La función $\alpha(n)$ está muy relacionada con una función de crecimiento muy rápido $A(x, y)$, conocida como *función de Ackermann*. $A(x, y)$ está definida en forma recursiva por:

$$A(0, y) = 1 \text{ para } y \geq 0$$

$$A(1, 0) = 2$$

$$A(x, 0) = x + 2 \text{ para } x \geq 2$$

$$A(x, y) = A(A(x-1, y), y-1) \text{ para } x, y \geq 1$$

Cada valor de y define una función de una variable. Por ejemplo, la tercera línea dice que para $y = 0$, esta función es «sumar 2». Para $y = 1$, se tiene $A(x, 1) = A(A(x-1, 1), 0) = A(x-1, 1) + 2$, para $x > 1$, con $A(1, 1) = A(A(0, 1), 0) = A(1, 0) =$

= 2. Así, $A(x, 1) = 2x$ para toda $x \geq 1$. En otras palabras, $A(x, 1)$ es «multiplicar por 2». Después, $A(x, 2) = A(A(x-1, 2), 1) = 2A(x-1, 2)$ para $x > 1$. También, $A(1, 2) = A(A(0, 2), 1) = A(1, 1) = 2$. Así, $A(x, 2) = 2^x$. De modo semejante, es posible demostrar que $A(x, 3) = 2^{2^{x-1}}$ (pila de x números 2), mientras que $A(x, 4)$ crece tan rápido que no existe ninguna notación matemática aceptada para tal función.

Una función de Ackermann de una sola variable puede definirse haciendo $A(x) = A(x, x)$. La función $\alpha(n)$ es una pseudoinversa de esta función de una sola variable. Esto es, $\alpha(n)$ es la menor x tal que $n \leq A(x)$. Por ejemplo, $A(1) = 2$, así $\alpha(1) = \alpha(2) = 1$. $A(2) = 4$, por lo que $\alpha(3) = \alpha(4) = 2$. $A(3) = 8$, de modo que $\alpha(5) = \dots = \alpha(8) = 3$. De lo anterior parece que $\alpha(n)$ crece bastante uniformemente.

Sin embargo, $A(4)$ es una pila de 65 536 números 2. Como $\log(A(4))$ es una pila de 65 535 números 2, no cabe esperar siquiera escribir $A(4)$ en forma explícita, ya que se necesitarían $\log(A(4))$ bits. Así, $\alpha(n) \leq 4$ para todos los enteros n que sea posible encontrar. Aun así, $\alpha(n)$ alcanzará finalmente los valores 5, 6, 7, ... en su curso inimaginablemente lento a infinito.

5.6 TDA con COMBINA y DIVIDE

Sea S un conjunto cuyos miembros están ordenados de acuerdo con la relación $<$. La operación $DIVIDE(S, S_1, S_2, x)$ divide a S en dos conjuntos: $S_1 = \{a \mid a \text{ está en } S \text{ y } a < x\}$ y $S_2 = \{a \mid a \text{ está en } S \text{ y } a \geq x\}$. El valor de S después de dividirse es indefinido, a menos que sea S_1 o S_2 . Existen varias situaciones donde es imprescindible la operación de división de conjuntos al comparar cada miembro con un valor fijo x . Se considerará uno de esos problemas aquí.

Problema de la subsecuencia común más larga

Una *subsecuencia* de una secuencia x se obtiene al eliminar cero o más elementos de x (no necesariamente contiguos). Dadas dos secuencias x e y , una *subsecuencia común más larga* (SCL) es una secuencia más larga que es una subsecuencia de x y de y .

Por ejemplo, una SCL de 1, 2, 3, 2, 4, 1, 2 y de 2, 4, 3, 1, 2, 1 es la subsecuencia 2, 3, 2, 1, formada como se muestra en la figura 5.28. Existen otras SCL también, como 2, 4, 1, 2, pero no existen subsecuencias comunes de longitud 5. Existe un mandato de UNIX, llamado *diff*, que compara archivos línea por línea y encuentra una subsecuencia común más larga, donde una línea de un archivo se considera como un elemento de la subsecuencia, esto es, las líneas completas son semejantes a los enteros 1, 2, 3 y 4 de la figura 5.28. La suposición que respalda al mandato *diff* es que las líneas de cada archivo que no se encuentran en esta SCL son líneas insertadas, suprimidas o modificadas al ir de un archivo al otro. Por ejemplo, si los dos archivos son versiones del mismo programa realizadas con varios días de diferencia, *diff* encontrará los cambios, con una alta probabilidad.

Es posible encontrar varias soluciones generales al problema SCL que funcionan en $O(n^2)$ pasos en secuencias de longitud n . El mandato *diff* usa una estrategia dife-

rente que funciona bien cuando los archivos no tienen demasiadas repeticiones de ninguna línea. Por ejemplo, los programas tenderán a tener líneas **begin** y **end** repetidas muchas veces, pero no es probable que otras líneas se repitan.

El algoritmo empleado por *diff* para encontrar una SCL hace uso de una realización eficiente de conjuntos con las operaciones COMBINA y DIVIDE, para trabajar en un tiempo $O(p \log n)$, donde n es el número máximo de líneas de un archivo y p es el número de pares de posiciones, una de cada archivo, que tienen la misma línea. Por ejemplo, el valor de p para las cadenas de la figura 5.28 es 12. Los dos unos de cada cadena contribuyen con cuatro pares, los números 2 contribuyen con seis pares, y 3 y 4 contribuyen con un par cada uno. En el peor caso, p podría ser n^2 , y este algoritmo llevaría un tiempo $O(n^2 \log n)$. Sin embargo, en la práctica, p suele estar más cercano a n , por lo que puede esperarse una complejidad de tiempo $O(n \log n)$.

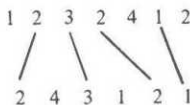


Fig. 5.28. Subsecuencia común más larga.

Para empezar la descripción del algoritmo, sean $A = a_1 a_2 \dots a_n$ y $B = b_1 b_2 \dots b_m$ las dos cadenas de las cuales se desea obtener la SCL. El primer paso es tabular las posiciones de la cadena A en las que aparezca a para cada valor de a . Esto es, se define $\text{LUGARES}(a) = \{i \mid a = a_i\}$. Se pueden calcular los conjuntos $\text{LUGARES}(a)$ al construir una correspondencia de símbolos a encabezados de las listas de posiciones. Por medio de una tabla de dispersión se pueden crear los conjuntos $\text{LUGARES}(a)$ con $O(n)$ «pasos» en promedio, donde un «paso» es el tiempo que lleva operar en un símbolo, por ejemplo, dispersarlo o compararlo con otro. Este tiempo podría ser una constante si los símbolos fueran caracteres o enteros. Sin embargo, si los símbolos de A y B son en realidad líneas de texto, los pasos llevan una cantidad de tiempo que depende de la longitud promedio de una línea de texto.

Al terminar el cálculo de $\text{LUGARES}(a)$ para cada símbolo a que aparece en una cadena A , es posible encontrar una SCL. Para simplificar las cosas, sólo se mostrará cómo encontrar la longitud de la SCL y se deja como ejercicio la construcción real de la SCL. El algoritmo considera cada b_j , para $j = 1, 2, \dots, m$, en orden. Después de considerar b_j , es necesario conocer la longitud de la SCL de las cadenas $a_1 \dots a_i$ y $b_1 \dots b_j$ para cada i entre 0 y n .

Se agrupan los valores de i en conjuntos S_k , para $k = 0, 1, \dots, n$, donde S_k consiste en todos los enteros i tales que la SCL de $a_1 \dots a_i$ y $b_1 \dots b_j$ tiene la longitud k . Obsérvese que S_k siempre será un conjunto de enteros consecutivos, y los enteros de S_{k+1} son mayores que los de S_k para toda k .

Ejemplo 5.8. Considérese la figura 5.28, con $j = 5$. Si se intenta comparar cero símbolos de la primera cadena con los cinco primeros de la segunda (24312), se tendrá una SCL de longitud 0, y así 0 está en S_0 . Si se emplea el primer símbolo de la primera cadena, es posible obtener una SCL de longitud 1, y si se manejan los dos pri-

meros símbolos, 12, una SCL de longitud 2. Sin embargo, al usar 123, los primeros tres símbolos, también se obtiene una SCL de longitud 2 cuando se comparan con 24312. Procediendo de esta manera, se descubre que $S_0 = \{0\}$, $S_1 = \{1\}$, $S_2 = \{2, 3\}$, $S_3 = \{4, 5, 6\}$ y $S_4 = \{7\}$. $\square$

Supóngase que se han calculado los S_k para la posición $j - 1$ de la segunda cadena y se desean modificar para aplicarlos a la posición j . Considérese el conjunto $LUGARES(b_j)$. Para cada r en $LUGARES(b_j)$, se ve si es posible aumentar alguna de las SCL al agregar el resultado de comparar a_r y b_j a la SCL de $a_1 \dots a_{r-1}$ y de $b_1 \dots b_j$. Esto es, si tanto $r - 1$ como r están en S_k , entonces toda $s \geq r$ en S_k pertenece en realidad a S_{k+1} cuando se toma en consideración b_j . Para ver esto, se observa que se pueden obtener k comparaciones entre $a_1 \dots a_{r-1}$ y $b_1 \dots b_{j-1}$, a las cuales se les agrega la comparación entre a_r y b_j . S_k y S_{k+1} se pueden modificar con los siguientes pasos.

1. ENCUENTRA(r) para obtener S_k .
2. Si ENCUENTRA($r - 1$) no está en S_k , entonces no se logra ninguna mejora de comparar b_j con a_r . Saltar los siguientes pasos y no modificar S_k o S_{k+1} .
3. Si ENCUENTRA($r - 1$) = S_k , aplicar DIVIDE(S_k , S_k , S'_k , r) para separar de S_k los miembros que sean mayores o iguales que r .
4. COMBINA (S'_k , S_{k+1} , S_{k+1}) para pasar esos elementos a S_{k+1} .

Es importante considerar primero los miembros más grandes de $LUGARES(b_j)$. Para ver por qué, supóngase, por ejemplo, que 7 y 9 pertenecen a $LUGARES(b_j)$ y que, antes de considerar b_j , $S_3 = \{6, 7, 8, 9\}$ y $S_4 = \{10, 11\}$.

Si se considera 7 antes que 9, se divide S_3 en $S_3 = \{6\}$ y $S'_3 = \{7, 8, 9\}$, entonces se hace $S_4 = \{7, 8, 9, 10, 11\}$. Si luego se considera 9, se divide S_4 en $S_4 = \{7, 8\}$ y $S'_4 = \{9, 10, 11\}$, para combinar 9, 10 y 11 en S_5 . Así, se ha pasado 9 de S_3 a S_5 considerando sólo una posición más en la segunda cadena, que representa una imposibilidad. Intuitivamente, lo que sucede es que se ha comparado en forma errónea b_j con a_7 y a_9 al crear una SCL imaginaria de longitud 5.

En la figura 5.29, se observa un bosquejo del algoritmo que genera los conjuntos S_k conforme se va analizando la segunda cadena. Para determinar la longitud de una SCL, sólo hay que ejecutar ENCUENTRA(n) al final.

procedure SCL;

begin

- (1) asigna valores iniciales $S_0 = \{0, 1, \dots, n\}$ y $S_i = \emptyset$ para $i = 1, 2, \dots, n$;
- (2) **for** $j = 1$ **to** n **do** { calcula los S_k en la posición j }
- (3) **for** r en $LUGARES(b_j)$, primero el mayor **do begin**
- (4) $k := ENCUENTRA(r)$;
- (5) **if** $k = ENCUENTRA(r-1)$ **then begin** { r no es el menor en S_k }
- (6) $DIVIDE(S_k, S_k, S'_k, r)$;
- (7) $COMBINA(S'_k, S_{k+1}, S_{k+1})$
- end**
- end**
- end;** { SCL }

Fig. 5.29. Esbozo del programa de subsecuencia común más larga.

Análisis de tiempo del algoritmo SCL

Como se mencionó, el algoritmo de la figura 5.29 es un enfoque útil sólo si no existen demasiadas comparaciones entre símbolos de las dos cadenas. La medida de número de comparaciones es

$$p = \sum_{j=1}^m |\text{LUGARES}(b_j)|$$

donde $|\text{LUGARES}(b_j)|$ denota el número de elementos en el conjunto $\text{LUGARES}(b_j)$. En otras palabras, p es la suma sobre todas las b_j del número de posiciones de la primera cadena que coinciden con b_j . Recuérdese que en el análisis de la comparación de archivos, se esperaba que p fuera del mismo orden que m y n , las longitudes de las dos cadenas (archivos).

Esto hace que el árbol 2-3 sea una buena estructura para los conjuntos S_k . Se puede asignar valor inicial a esos conjuntos, como en la línea (1) de la figura 5.29, en $O(n)$ pasos. La operación **ENCUENTRA** requiere un arreglo que sirva como una correspondencia de las posiciones r a las hojas de r y también requiere apuntadores a los padres en el árbol 2-3. El nombre del conjunto, es decir, k para S_k , puede conservarse en la raíz, así que se puede ejecutar **ENCUENTRA** en $O(\log n)$ pasos, siguiendo los apuntadores a los padres hasta llegar a la raíz. Así, el conjunto de las ejecuciones de las líneas (4) y (5) juntas lleva un tiempo $O(p \log n)$, pues dichas líneas se ejecutan una a la vez, para cada coincidencia encontrada.

La operación **COMBINA** de la línea (5) tiene la propiedad especial de que cada miembro de S'_k es menor que cada miembro de S_{k+1} , y se puede aprovechar este hecho cuando se usan árboles 2-3 para la aplicación $\dagger$. Para empezar la operación **COMBINA**, se coloca el árbol 2-3 de S'_k a la izquierda del de S_{k+1} . Si ambos tienen la misma altura, se crea una raíz nueva con las raíces de los dos árboles como sus hijos. Si S'_k es más corto, se inserta la raíz de ese árbol como el hijo más a la izquierda del nodo más a la izquierda de S_{k+1} en el nivel apropiado. Si este nodo tiene ahora cuatro hijos, se modifica el árbol exactamente de la misma forma que se hizo en el procedimiento **INSERTAR** de la figura 5.20. En la figura 5.30 se muestra un ejemplo. En forma semejante, si S_{k+1} es más corto, se hace que su raíz quede como el hijo más a la derecha del nodo más a la derecha de S'_k en el nivel apropiado.

La operación **DIVIDE** en r requiere subir en el árbol a partir de una hoja r , duplicar cada nodo interior del camino y dar una copia para cada uno de los dos árboles resultantes. Los nodos sin hijos se eliminan, y los nodos con un hijo se retiran para que ese hijo quede insertado en el árbol y nivel adecuados.

Ejemplo 5.9. Supóngase que se divide el árbol de la figura 5.30(b) en el nodo 9. Los dos árboles con nodos duplicados se muestran en la figura 5.31(a). A la izquierda, el padre de 8 tiene sólo un hijo, por lo que 8 se convierte en hijo del padre de 6 y 7.

$\dagger$ Estrictamente hablando, se debería usar un nombre diferente para la operación **COMBINA**, pues la realización que se propone no efectúa la unión arbitraria de conjuntos disjuntos, y hace que los elementos estén clasificados para que puedan llevarse a cabo operaciones como **DIVIDE** y **ENCUENTRA**.

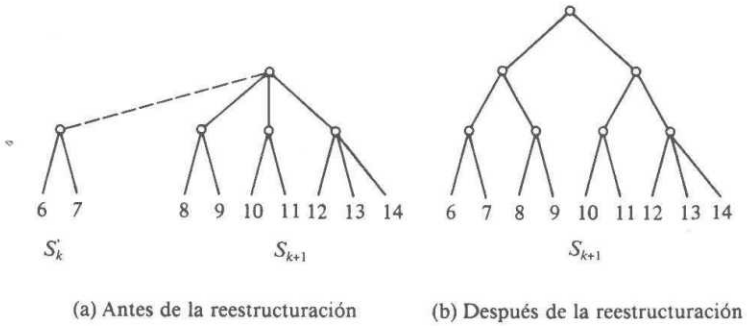


Fig. 5.30. Ejemplo de COMBINA.

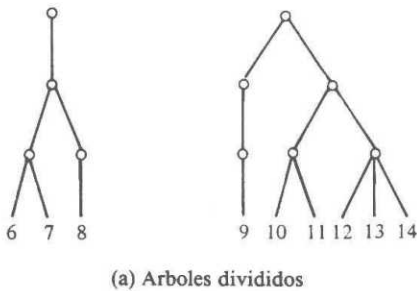


Fig. 5.31. Ejemplo de DIVIDE.

Este padre tiene ahora tres hijos, así que todo queda como debe ser; si tuviera cuatro hijos, se habría creado un nodo nuevo e insertado en el árbol. Sólo es necesario eliminar nodos con cero hijos (el padre anterior de 8) y la cadena de nodos con un

hijo que llega a la raíz. El padre de 6, 7 y 8 se convierte en la nueva raíz, como se muestra en la figura 5.31(b). En forma semejante, en el árbol del lado derecho, 9 queda como hermano de 10 y 11, y se eliminan los nodos innecesarios, como se muestra también en la figura 5.31(b). $\square$

Si se divide y reorganiza el árbol 2-3 de abajo hacia arriba, considerando una gran cantidad de casos es posible demostrar que $O(\log n)$ pasos son suficientes. Así, el tiempo total consumido en las líneas (6) y (7) de la figura 5.29 es $O(p \log n)$, y el algoritmo en su totalidad requiere $O(p \log n)$ pasos. Es necesario agregar el tiempo de preprocesamiento requerido para calcular y clasificar $LUGARES(a)$ para los símbolos a . Como ya se mencionó, si los símbolos a son objetos «grandes», este tiempo puede ser mucho mayor que cualquier otra parte del algoritmo. Como se verá en el capítulo 8, si los símbolos pueden ser manipulados y comparados en «pasos» sencillos, un tiempo $O(n \log n)$ es suficiente para clasificar la primera cadena $a_1 a_2 \dots a_n$ (en realidad, para clasificar los objetos (i, a_i) en el segundo campo), así que $LUGARES(a)$ puede leerse en esta lista en un tiempo $O(n)$. Así, la longitud de la SCL puede calcularse en un tiempo $O(\max(n, p) \log n)$, el cual puede considerarse como $O(p \log n)$, ya que $p \geq n$ es normal.

Ejercicios

- 5.1 Dibújense todos los posibles árboles binarios de búsqueda que contengan los elementos 1, 2, 3 y 4.
- 5.2 Insértense los enteros 7, 2, 9, 0, 5, 6, 8, 1 en un árbol binario de búsqueda por medio de la aplicación repetida del procedimiento INSERTA de la figura 5.3.
- 5.3 Muéstrase el resultado obtenido al suprimir 7 y después 2 del árbol final del ejercicio 5.2.
- *5.4 Cuando se eliminan dos elementos de un árbol binario de búsqueda con el procedimiento de la figura 5.5, ¿depende alguna vez el árbol final del orden en que se eliminaron?
- 5.5 Se desea tener información de todas las subcadenas de cinco caracteres que ocurren dentro de una cadena dada por medio de un trie. Muéstrase el trie que resulta de insertar las 14 subcadenas de longitud 5 de la cadena ABCDABACDEBACADEBA.
- *5.6 Para realizar el ejercicio 5.5, se podría poner un apuntador en cada hoja, lo cual representaría la cadena *abcde*, al nodo interior que representa el sufijo *bcd*. De esa manera, si se recibe el siguiente símbolo, por ejemplo *F*, no hay que insertar todo *bcd*, partiendo de la raíz. Más aún, habiendo visto *abcde*, es posible crear también nodos para *bcd*, *cde*, *de*, y *e*, ya que, a menos que la secuencia concluya abruptamente, más tarde se necesitarán esos nodos. Modifíquense la estructura de datos del trie para colocar esos apun-

tadores, y el algoritmo de inserción en un trie, para aprovechar esta estructura de datos.

- 5.7 Muéstrase el árbol 2-3 que resulta de insertar los elementos 5, 2, 7, 0, 3, 4, 6, 1, 8, 9 en un conjunto vacío, representado como un árbol 2-3.
- 5.8 Muéstrase el resultado obtenido de eliminar el 3 del árbol 2-3 del ejercicio 5.7.
- 5.9 Muéstranse los valores sucesivos de las S_i al aplicar el algoritmo SCL de la figura 5.29 con la primera cadena *abacabada*, y la segunda *bdbachad*.
- 5.10 Supóngase que se emplean árboles 2-3 para aplicar las operaciones COMBINA y DIVIDE de la sección 5.8.
 - a) Muéstrase el resultado de dividir el árbol del ejercicio 5.7 en el elemento 6.
 - b) Combínese el árbol del ejercicio 5.7 con el árbol que consiste en hojas para los elementos 10 y 11.
- 5.11 Algunas estructuras estudiadas en este capítulo pueden modificarse fácilmente para manejar el TDA CORRESPONDENCIA. Escribanse los procedimientos ANULA, ASIGNA y CALCULA para operar sobre las siguientes estructuras de datos.
 - a) Árboles binarios de búsqueda. El ordenamiento «<» se aplica a los elementos del dominio.
 - b) Árboles 2-3. En los nodos interiores, colóquese sólo el campo clave de los elementos del dominio.
- 5.12 Demuéstrase que en cualquier subárbol de un árbol binario de búsqueda, el elemento mínimo es un nodo sin hijo izquierdo.
- 5.13 Empléese el ejercicio 5.12 para producir una versión no recursiva de SUPRIME_MIN.
- 5.14 Escribanse los procedimientos ASIGNA, VALOR_DE, ANULA, y TOMA_NUEVO para nodos de tries representados como listas de celdas.
- *5.15 ¿Cómo se comparan el trie (con la realización de lista de celdas), la tabla de dispersión abierta y el árbol binario de búsqueda en cuanto a velocidad y utilización de espacio cuando los elementos son cadenas de hasta diez caracteres?
- *5.16 Si los elementos de un conjunto se ordenan de acuerdo con la relación «<», se pueden guardar uno o dos elementos (no sólo sus claves) en los nodos internos de un árbol 2-3, sin tener que guardarlos en las hojas. Escribanse los procedimientos INSERTA y SUPRIME para árboles 2-3 de este tipo.
- 5.17 Otra modificación que se podría hacer a los árboles 2-3 sería guardar sólo claves en nodos internos, sin requerir que las claves k_1 y k_2 de un nodo sean

en realidad las claves mínimas del segundo y tercer subárboles, sino sólo que todas las claves k del tercer subárbol satisfagan $k \geq k_2$, todas las claves del segundo satisfagan $k_1 \leq k < k_2$, y todas las claves k del primero satisfagan $k < k_1$.

- a) ¿Cómo se simplifica SUPRIME con esta convención?
- b) ¿Qué operaciones de diccionarios o de correspondencias se hacen más complicadas o menos eficientes?

***5.18** Otra estructura de datos que maneja diccionarios con la operación MIN es el *árbol AVL* (nombre debido a las iniciales de sus inventores) o *árbol balanceado por altura*. Son árboles binarios de búsqueda en los cuales no se permite que las alturas de dos hermanos difieran en más de uno. Escribanse los procedimientos INSERTA y SUPRIME respetando la propiedad de árbol AVL.

5.19 Escribese un programa en Pascal para el procedimiento *inserta1* de la figura 5.21.

***5.20** Un *autómata finito* consiste en un conjunto de estados, los cuales se tomarán como los enteros $1..n$, y una tabla *transiciones [estado, entrada]* que da el *siguiente estado* para cada *estado* y cada carácter de *entrada*. En este caso, se supondrá que la entrada es 0 ó 1; más aún, ciertos estados se designan como *estados de aceptación*. También se supondrá que todos, y únicamente los estados con numeración par son de aceptación. Dos estados p y q son *equivalentes* si son el mismo o a) ambos son de aceptación o ambos son de no aceptación, b) con la entrada 0 se transfieren a estados equivalentes, y c) con la entrada 1 se transfieren a estados equivalentes. Intuitivamente, los estados equivalentes se comportan igual en todas las secuencias de entrada. Escribese un programa que use las operaciones de CONJUNTO_CE y que calcule los conjuntos de estados equivalentes de un *autómata finito* dado.

****5.21** En la realización con árboles de CONJUNTO_CE:

- a) Demuéstrese que se necesita un tiempo $\Omega(n \log n)$ para ciertas listas de n operaciones si se usa la compresión de caminos, pero se permite la combinación de árboles grandes con otros más pequeños.
- b) Demuéstrese que $O(n \alpha(n))$ es el tiempo de ejecución en el peor caso para n operaciones si se usa compresión de caminos, y se combina siempre el árbol más pequeño con el más grande.

5.22 Seleccíonese una estructura de datos y escribese un programa para calcular LUGARES (tal como se definió en la Sec. 5.6) en un tiempo promedio $O(n)$ para cadenas de longitud n .

***5.23** Modifíquese el procedimiento SCL de la figura 5.29 para obtener la SCL, no sólo su longitud.

- *5.24 Escribase en forma detallada un procedimiento **DIVIDE** para trabajar con árboles 2-3.
- *5.25 Si los elementos de un conjunto representado por medio de un árbol 2-3 constaran sólo de un campo clave, un elemento cuya clave apareciera en un nodo interno no necesitaría estar en una hoja. Escribanse otra vez las operaciones de diccionario para aprovechar este hecho y evitar el almacenamiento de un elemento en dos nodos diferentes.

Notas bibliográficas

Los tries fueron propuestos por primera vez por Fredkin [1960]. Bayer y McCreight [1972] introdujeron los árboles B que, como se analizará en el capítulo 11, son una generalización de los árboles 2-3. El primer uso que se dio a los árboles 2-3 se debió a J. E. Hopcroft en 1970 (no publicado) para inserción, eliminación, concatenación y división, y a Ullman [1974] para un problema de optimización de código.

La estructura del árbol de la sección 5.5, que emplea compresión de caminos y combinación del menor con el mayor, fue utilizada primero por M. D. McIlroy y R. Morris para construir árboles abarcadores de costo mínimo. El rendimiento de la realización con árboles de los **CONJUNTO_CE** fue analizada por Fischer [1972] y por Hopcroft y Ullman [1973]. El ejercicio 5.21 es de Tarjan [1974].

La solución al problema de la **SCL** de la sección 5.6 es de Hunt y Szymanski [1975]. Una estructura de datos eficiente para **ENCUENTRA**, **DIVIDE** y **COMBINA** restringido (donde los elementos de un conjunto son menores que los del otro) se describe en Van Emde Boas, Kaas y Zijlstra [1975].

El ejercicio 5.6 está basado en un algoritmo eficiente para comparación de patrones desarrollado por Weiner [1973]. La variación del árbol 2-3 del ejercicio 5.16 se comenta con detalle en Wirth [1976]. La estructura de árbol **AVL** del ejercicio 5.18 es de Adel'son-Vel'skii y Landis [1962].

6

Grafos dirigidos

En los problemas originados en ciencias de la computación, matemáticas, ingeniería y muchas otras disciplinas, a menudo es necesario representar relaciones arbitrarias entre objetos de datos. Los grafos dirigidos y los no dirigidos son modelos naturales de tales relaciones. Este capítulo presenta las estructuras de datos básicas que pueden usarse para representar grafos dirigidos. También se presentan algunos algoritmos básicos para la determinación de conectividad en grafos dirigidos y para encontrar los caminos más cortos.

6.1 Definiciones fundamentales

Un *grafo dirigido* G consiste en un conjunto de vértices V y un conjunto de arcos A . Los vértices se denominan también *nodos* o *puntos*; los arcos pueden llamarse *arcos dirigidos* o *líneas dirigidas*. Un arco es un par ordenado de vértices (v, w) ; v es la *cola* y w la *cabeza* del arco. El arco (v, w) se expresa a menudo como $v \rightarrow w$ y se representa como



Obsérvese que la «punta de la flecha» está en el vértice llamado «cabeza», y la cola, en el vértice llamado «cola». Se dice que el arco $v \rightarrow w$ *va de v a w*, y que w es *adyacente a v*.

Ejemplo 6.1. La figura 6.1 muestra un grafo dirigido con cuatro vértices y cinco arcos. □

Los vértices de un grafo dirigido pueden usarse para representar objetos, y los arcos, relaciones entre los objetos. Por ejemplo, los vértices pueden representar ciudades, y los arcos, vuelos aéreos de una ciudad a otra. En otro ejemplo, como el que se presentó en la sección 4.2, un grafo dirigido puede emplearse para representar el flujo de control en un programa de computador. Los vértices representan bloques básicos, y los arcos, posibles transferencias del flujo de control.

Un *camino* en un grafo dirigido es una secuencia de vértices $v_1, v_2, \dots, v_n$, tal que $v_1 \rightarrow v_2, v_2 \rightarrow v_3, \dots, v_{n-1} \rightarrow v_n$ son arcos. Este camino *va del* vértice v_1 al vértice v_n .

pasa por los vértices $v_2, v_3, \dots, v_{n-1}$, y termina en el vértice v_n . La *longitud* de un camino es el número de arcos en ese camino, en este caso, $n - 1$. Como caso especial, un vértice sencillo, v , por sí mismo denota un camino de longitud cero de v a v . En la figura 6.1, la secuencia 1, 2, 4 es un camino de longitud 2 que va del vértice 1 al vértice 4.

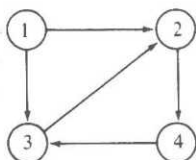


Fig. 6.1. Grafo dirigido.

Un camino es *simple* si todos sus vértices, excepto tal vez el primero y el último, son distintos. Un *ciclo* simple es un camino simple de longitud por lo menos uno, que empieza y termina en el mismo vértice. En la figura 6.1, el camino 3, 2, 4, 3 es un ciclo de longitud 3.

En muchas aplicaciones es útil asociar información a los vértices y arcos de un grafo dirigido. Para este propósito es posible usar un *grafo dirigido etiquetado*, en el cual cada arco, cada vértice o ambos pueden tener una etiqueta asociada. Una etiqueta puede ser un nombre, un costo o un valor de cualquier tipo de datos dado.

Ejemplo 6.2. La figura 6.2 muestra un grafo dirigido etiquetado en el que cada arco está etiquetado con una letra que causa una transición de un vértice a otro. Este grafo dirigido etiquetado tiene la interesante propiedad de que las etiquetas de los arcos de cualquier ciclo que sale del vértice 1 y vuelve a él producen una cadena de caminos a y b en la cual los números de a y de b son pares. □

En un grafo dirigido etiquetado, un vértice puede tener a la vez un nombre y una etiqueta. A menudo se empleará la etiqueta del vértice como si fuera el nombre. Así, los números de la figura 6.2 pueden interpretarse como nombres o como etiquetas de vértices.

6.2 Representaciones de grafos dirigidos

Para representar un grafo dirigido se pueden emplear varias estructuras de datos; la selección apropiada depende de las operaciones que se aplicarán a los vértices y a los arcos del grafo. Una representación común para un grafo dirigido $G = (V, A)$ es la *matriz de adyacencia*. Supóngase que $V = \{1, 2, \dots, n\}$. La matriz de adyacencia para G es una matriz A de dimensión $n \times n$, de elementos booleanos, donde $A[i, j]$ es verdadero si, y sólo si, existe un arco que vaya del vértice i al j . Con frecuencia se exhibirán matrices de adyacencias con 1 para verdadero y 0 para falso; las matrices de adyacencias pueden incluso obtenerse de esa forma. En la representación con una

matriz de adyacencia, el tiempo de acceso requerido a un elemento es independiente del tamaño de V y A . Así, la representación con matriz de adyacencia es útil en los algoritmos para grafos, en los cuales suele ser necesario saber si un arco dado está presente.

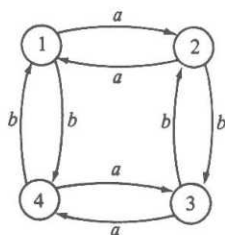


Fig. 6.2. Grafo dirigido de transiciones.

Algo muy relacionado con esto es la representación con *matriz de adyacencia etiquetada* de un grafo dirigido, donde $A[i, j]$ es la etiqueta del arco que va del vértice i al vértice j . Si no existe un arco de i a j , debe emplearse como entrada para $A[i, j]$ un valor que no pueda ser una etiqueta válida.

Ejemplo 6.3 La figura 6.3. muestra la matriz de adyacencia etiquetada para el grafo dirigido de la figura 6.2. Aquí, el tipo de la etiqueta es un carácter, y un espacio representa la ausencia de un arco. $\square$

	1	2	3	4
1		a		b
2	a		b	
3		b		a
4	b		a	

Fig. 6.3. Matriz de adyacencia etiquetada para el grafo dirigido de la figura 6.2.

La principal desventaja de usar una matriz de adyacencia para representar un grafo dirigido es que requiere un espacio $\Omega(n^2)$ aun si el grafo dirigido tiene menos de n^2 arcos. Sólo leer o examinar la matriz puede llevar un tiempo $O(n^2)$, lo cual invalidaría los algoritmos $O(n)$ para la manipulación de grafos dirigidos con $O(n)$ arcos.

Para evitar esta desventaja, se puede utilizar otra representación común para un grafo dirigido $G = (V, A)$ llamada representación con *lista de adyacencia*. La lista de adyacencia para un vértice i es una lista, en algún orden, de todos los vértices adyacentes a i . Se puede representar G por medio de un arreglo *CABEZA*, donde *CABEZA*[i] es un apuntador a la lista de adyacencia del vértice i . La representación con lista de adyacencia de un grafo dirigido requiere un espacio proporcional a la suma del número de vértices más el número de arcos; se usa bastante cuando

el número de arcos es mucho menor que n^2 . Sin embargo, una desventaja potencial de la representación con lista de adyacencia es que puede llevar un tiempo $O(n)$ determinar si existe un arco del vértice i al vértice j , ya que puede haber $O(n)$ vértices en la lista de adyacencia para el vértice i .

Ejemplo 6.4. La figura 6.4 muestra una representación con lista de adyacencia para el grafo dirigido de la figura 6.1, donde se usan listas enlazadas sencillas. Si los arcos tienen etiquetas, éstas podrían incluirse en las celdas de la lista ligada.

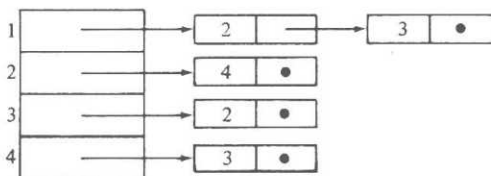


Fig. 6.4. Representación con lista de adyacencia para el grafo dirigido de la figura 6.1

Si hubo inserciones y supresiones en las listas de adyacencias, sería preferible tener el arreglo *CABEZA* apuntando a celdas de encabezamiento que no contienen vértices adyacentes †. Por otra parte, si se espera que el grafo permanezca fijo, sin cambios (o con muy pocos) en las listas de adyacencias, sería preferible que *CABEZA*[i] fuera un cursor a un arreglo *ADY*, donde *ADY*[*CABEZA*[i]], *ADY*[*CABEZA*[i] + 1], ..., y así sucesivamente, contuvieran los vértices adyacentes al vértice i , hasta el punto en *ADY* donde se encuentra por primera vez un cero, el cual marca el fin de la lista de vértices adyacentes a i . Por ejemplo, la figura 6.1 puede representarse con la figura 6.5. □

TDA grafo dirigido

Se podría definir un TDA que correspondiera formalmente al grafo dirigido y estudiar las implantaciones de sus operaciones. No se redundará demasiado en esto, porque hay poco material en verdad nuevo y las principales estructuras de datos para grafos ya han sido cubiertas. Las operaciones más comunes en grafos dirigidos incluyen la lectura de la etiqueta de un vértice o un arco, la inserción o supresión de vértices y arcos, y el recorrido de arcos desde la cola hasta la cabeza.

Las últimas operaciones requieren más cuidado. Con frecuencia, se encuentran en proposiciones informales de programas como

for cada vértice w adyacente al vértice v do
 {alguna acción sobre w } (6.1)

† Esta es otra manifestación del viejo problema de Pascal de hacer inserción y supresión en posiciones arbitrarias de listas enlazadas sencillas.

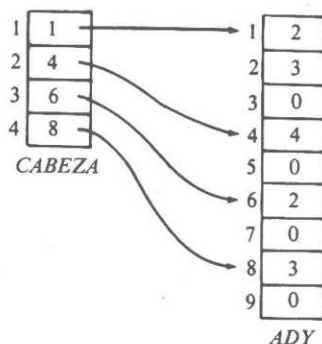


Fig. 6.5. Otra representación con lista de adyacencia de la figura 6.1.

Para obtener esto, es necesaria la noción de un tipo *índice* para el conjunto de vértices adyacentes a algún vértice v . Por ejemplo, si las listas de adyacencias se usan para representar el grafo, un índice es, en realidad, una posición en la lista de adyacencia de v . Si se usa una matriz de adyacencia, un índice es un entero que representa un vértice adyacente. Se requieren las tres operaciones siguientes en grafos dirigidos.

1. **PRIMERO**(v), que devuelve el índice del primer vértice adyacente a v . Se devuelve un vértice nulo Λ si no existe ningún vértice adyacente a v .
2. **SIGUIENTE**(v, i), que devuelve el índice posterior al índice i de los vértices adyacentes a v . Se devuelve Λ si i es el último vértice de los vértices adyacentes a v .
3. **VERTICE**(v, i), que devuelve el vértice cuyo índice i está entre los vértices adyacentes a v .

Ejemplo 6.5. Si se escoge la representación con matriz de adyacencia, **VERTICE**(v, i) devuelve i . **PRIMERO**(v) y **SIGUIENTE**(v, i) pueden escribirse como en la figura 6.6, para operar en una matriz booleana A de $n \times n$ definida de manera externa. Se supone que A está declarada como

array [1.. n , 1.. n] **of** boolean

y que 0 se emplea para Λ . Después, se obtiene la proposición (6.1) como en la figura 6.7. $\square$

```
function PRIMERO (  $v$ : integer ) : integer;
var
     $i$ : integer;
```

```

begin
  for  $i := 1$  to  $n$  do
    if  $A[v, i]$  then
      return( $i$ );
    return (0) { si se llega aquí,  $v$  no tiene vértices adyacentes }
end; { PRIMERO }

function SIGUIENTE (  $v$ : integer;  $i$ : integer ) : integer;
var
   $j$ : integer;
begin
  for  $j := i+1$  to  $n$  do
    if  $A[v, j]$  then
      return ( $j$ );
  return (0)
end; { SIGUIENTE }

```

Fig. 6.6. Operaciones para recorrer vértices adyacentes.

```

 $i := \text{PRIMERO}(v)$ ;
while  $i \neq \wedge$  do begin
   $w := \text{VERTICE}(v, i)$ ;
  { alguna acción en  $w$  }
   $i := \text{SIGUIENTE}(v, i)$ 
end

```

Fig. 6.7. Iteración en vértices adyacentes a v .

6.3 Problema de los caminos más cortos con un solo origen

En esta sección se considera un problema común de búsqueda de caminos en grafos dirigidos. Supóngase un grafo dirigido $G = (V, A)$ en el cual cada arco tiene una etiqueta no negativa, y donde un vértice se especifica como *origen*. El problema es determinar el costo del camino más corto del origen a todos los demás vértices de V , donde la *longitud de un camino* es la suma de los costos de los arcos del camino. Esto se conoce con el nombre de problema de *los caminos más cortos con un solo origen* †. Obsérvese que se hablará de caminos con «longitud» aun cuando los costos representan algo diferente, como tiempo.

Sea G un mapa de vuelos en el cual cada vértice representa una ciudad, y cada

† Se puede esperar que un problema más natural sea encontrar el camino más corto entre el origen y un vértice *destino* particular. Sin embargo, ese problema parece tan difícil en general como el de los caminos más cortos con un solo origen (a menos que se tenga la suerte de encontrar el camino al destino antes que alguno de los otros vértices y así terminar el algoritmo un poco antes que si se buscaran los caminos hacia todos los vértices).

arco $v \rightarrow w$, una ruta aérea de la ciudad v a la ciudad w . La etiqueta del arco $v \rightarrow w$ es el tiempo que se requiere para volar de v a w †. La solución del problema de los caminos más cortos con un solo origen para este grafo dirigido determinaría el tiempo de viaje mínimo para ir de cierta ciudad a todas las demás del mapa.

Para resolver este problema se manejará una técnica «ávida» conocida como *algoritmo de Dijkstra*, que opera a partir de un conjunto S de vértices cuya distancia más corta desde el origen ya es conocida. En principio, S contiene sólo el vértice de origen. En cada paso, se agrega algún vértice restante v a S , cuya distancia desde el origen es la más corta posible. Suponiendo que todos los arcos tienen costo no negativo, siempre es posible encontrar un camino más corto entre el origen y v que pasa sólo a través de los vértices de S , y que se llama *especial*. En cada paso del algoritmo, se utiliza un arreglo D para registrar la longitud del camino especial más corto a cada vértice. Una vez que S incluye todos los vértices, todos los caminos son «especiales», así que D contendrá la distancia más corta del origen a cada vértice.

El algoritmo se da en la figura 6.8, donde se supone que existe un grafo dirigido $G = (V, A)$ en el que $V = \{1, 2, \dots, n\}$ y el vértice 1 es el origen. C es un arreglo bidimensional de costos, donde $C[i, j]$ es el costo de ir del vértice i al vértice j por el arco $i \rightarrow j$. Si no existe el arco $i \rightarrow j$, se supone que $C[i, j]$ es ∞ , un valor mucho mayor que cualquier costo real. En cada paso, $D[i]$ contiene la longitud del camino especial más corto actual para el vértice i .

Ejemplo 6.6. Aplíquese *Dijkstra* al grafo dirigido de la figura 6.9. En principio, $S = \{1\}$, $D[2] = 10$, $D[3] = \infty$, $D[4] = 30$ y $D[5] = 100$. En la primera iteración del ciclo **for** de las líneas (4) a (8), $w = 2$ se selecciona como el vértice con el mínimo valor D . Después se hace $D[3] = \min(\infty, 10 + 50) = 60$. $D[4]$ y $D[5]$ no cambian porque el camino para llegar a ellos directamente desde 1 es más corto que pasar por el vértice 2. La secuencia de valores D después de cada iteración se muestra en la figura. 6.10. □

Para reconstruir el camino más corto del origen a cada vértice, se agrega otro arreglo P de vértices, tal que $P[v]$ contenga el vértice inmediato anterior a v en el camino más corto. Se asigna $P[v]$ valor inicial 1 para toda $v \neq 1$. El arreglo P puede actualizarse después de la línea (8) de *Dijkstra*. Si $D[w] + C[w, v] < D[v]$ en la línea (8), después se hace $P[v] := w$. Al término de *Dijkstra*, el camino a cada vértice puede encontrarse regresando por los vértices predecesores del arreglo P .

procedure *Dijkstra*;

{ *Dijkstra* calcula el costo de los caminos más cortos entre el vértice 1
y todos los demás de un grafo dirigido }

begin

- (1) $S := \{1\};$
- (2) **for** $i := 2$ **to** n **do**

† Cabría suponer que puede usarse un grafo no dirigido, ya que la etiqueta de los arcos $v \rightarrow w$ y $w \rightarrow v$ sería la misma. Sin embargo, los tiempos de viaje son diferentes en direcciones diferentes, debido a los vientos. De cualquier forma, aunque las etiquetas $v \rightarrow w$ y $w \rightarrow v$ fueran idénticas, esto no ayudaría a resolver el problema.

```

(3)       $D[i] := C[1, i];$  { asigna valor inicial a  $D$  }
(4)  for  $i := 1$  to  $n-1$  do begin
(5)      elige un vértice  $w$  en  $V-S$  tal que  $D[w]$  sea un mínimo;
(6)      agrega  $w$  a  $S$ ;
(7)      for cada vértice  $v$  en  $V-S$  do
(8)           $D[v] := \min(D[v], D[w] + C[w, v])$ 
      end
end; { Dijkstra }
    
```

Fig. 6.8. Algoritmo de Dijkstra.

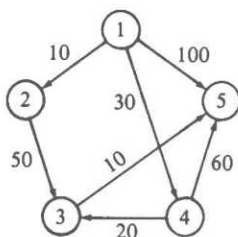


Fig. 6.9. Grafo dirigido con arcos etiquetados.

Ejemplo 6.7. Para el grafo dirigido del ejemplo 6.6, el arreglo P debe tener los valores $P[2] = 1$, $P[3] = 4$, $P[4] = 1$ y $P[5] = 3$. Para encontrar el camino más corto del vértice 1 al vértice 5, por ejemplo, se siguen los predecesores en orden inverso comenzando en 5. A partir del arreglo P , se determina que 3 es el predecesor de 5, 4 el predecesor de 3 y 1 el predecesor de 4. Así, el camino más corto entre los vértices 1 y 5 es 1, 4, 3, 5. □

Iteración	S	w	$D[2]$	$D[3]$	$D[4]$	$D[5]$
inicial	{1}	—	10	∞	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

Fig. 6.10. Cálculos de Dijkstra en el grafo dirigido de la figura 6.9.

Por qué funciona el algoritmo de Dijkstra

El algoritmo de Dijkstra es un ejemplo donde la «avidez» funciona, en el sentido de que lo que aparece localmente como lo mejor, se convierte en lo mejor de todo. En este caso, lo «mejor» localmente es encontrar la distancia al vértice w que

está fuera de S , pero tiene el camino especial más corto. Para ver por qué en este caso no puede haber un camino no especial más corto desde el origen hasta w , obsérvese la figura 6.11, en la que se muestra un camino hipotético más corto a w que primero sale de S para ir al vértice x , después (tal vez) entra y sale de S varias veces antes de llegar finalmente a w .

Pero si este camino es más corto que el camino especial más corto a w , el segmento inicial del camino entre el origen y x es un camino especial a x más corto que el camino especial más corto a w . (Obsérvese lo importante que es el hecho de que los costos no sean negativos; sin ello, este argumento no sería válido, y de hecho, el algoritmo de Dijkstra no funcionaría correctamente.) En este caso, se debió seleccionar x en vez de w en la línea (5) de la figura 6.8, porque $D[x]$ fue menor que $D[w]$.

Para completar la demostración de que la figura 6.8 funciona, se verifica que en todos los casos $D[v]$ es realmente la distancia más corta de un camino especial al vértice v . La clave de este razonamiento está en observar que al agregar un nuevo vértice w a S en la línea (6), las líneas (7) y (8) ajustan D para tener en cuenta la posibilidad de que exista ahora un camino especial más corto a v a través de w . Si ese camino va a través del anterior S a w , e inmediatamente después a v , su costo, $D[w] + C[w, v]$, será comparado con $D[v]$ en la línea (8), y $D[v]$ se reducirá si el nuevo camino especial es más corto. La otra posibilidad de un camino especial más corto se muestra en la figura 6.12, donde el camino va a w , después regresa al anterior S , a algún miembro x del S anterior, y luego a v .

Pero en realidad no puede existir tal camino; puesto que x se colocó en S antes que w , el más corto de todos los caminos entre el origen y x pasa sólo a través del S anterior. Por tanto, el camino hacia x a través de w , mostrado en la figura 6.12, no es más corto que el camino que va directo a x a través de S . Como resultado, la longitud del camino de la figura 6.12 entre el origen y w , x , y v no es menor que el anterior valor de $D[v]$, ya que $D[v]$ no fue mayor que la longitud del camino más corto hasta x a través de S y después directamente a w . Así, $D[v]$ no puede reducirse en la línea (8) por medio de un camino que pase por w y x , como el de la figura 6.12, y no es necesario considerar la longitud de esos caminos.

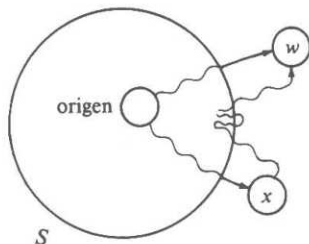


Fig. 6.11. Camino hipotético más corto a w .

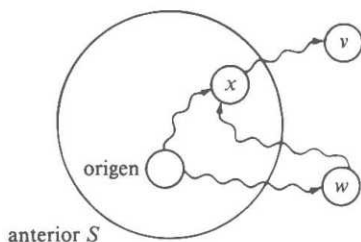


Fig. 6.12. Camino especial más corto imposible.

Tiempo de ejecución del algoritmo de Dijkstra

Supóngase que la figura 6.8 opera en un grafo dirigido con n vértices y a aristas. Si se emplea una matriz de adyacencia para representar el grafo dirigido, el ciclo de las líneas (7) y (8) lleva un tiempo $O(n)$, y se ejecuta $n - 1$ veces para un tiempo total de $O(n^2)$. El resto del algoritmo, como se puede observar, no requiere más tiempo que esto.

Si a es mucho menor que n^2 , puede ser mejor utilizar una representación con lista de adyacencia del grafo dirigido y emplear una cola de prioridad obtenida a manera de árbol parcialmente ordenado para organizar los vértices de $V - S$. El ciclo de las líneas (7) y (8) se puede realizar recorriendo la lista de adyacencia para w y actualizando las distancias en la cola de prioridad. Se hará un total de a actualizaciones, cada una con un costo de tiempo $O(\log n)$, por lo que el tiempo total consumido en las líneas (7) y (8) es ahora $O(a \log n)$, en vez de $O(n^2)$.

Las líneas (1) a (3) llevan un tiempo $O(n)$, al igual que las líneas (4) y (6). Al manejar la cola de prioridad para representar $V - S$, las líneas (5) y (6) implantan exactamente la operación SUPRIME_MIN y cada una de las $n - 1$ iteraciones de esas líneas requiere un tiempo $O(\log n)$.

Como resultado, el tiempo total consumido en esta versión del algoritmo de Dijkstra está acotado por $O(a \log n)$. Este tiempo de ejecución es mucho mejor que $O(n^2)$ si a es muy pequeña, comparada con n^2 .

6.4 Problema de los caminos más cortos entre todos los pares

Supóngase que se tiene un grafo dirigido etiquetado que da el tiempo de vuelo para ciertas rutas entre ciudades, y se desea construir una tabla que brinde el menor tiempo requerido para volar entre dos ciudades cualesquiera. Este es un ejemplo del problema de *los caminos más cortos entre todos los pares* (CMCP). Para plantear el problema con precisión, se emplea un grafo dirigido $G = (V, A)$ en el cual cada arco $v \rightarrow w$ tiene un costo no negativo $C[v, w]$. El problema CMCP es encontrar el camino de longitud más corta entre v y w para cada par ordenado de vértices (v, w) .

Podría resolverse este problema por medio del algoritmo de Dijkstra, tomando por turno cada vértice como vértice origen, pero una forma más directa de solución es mediante el algoritmo creado por R. W. Floyd. Por conveniencia, se supone otra vez que los vértices en v están numerados 1, 2, ..., n . El algoritmo de Floyd usa una matriz A de $n \times n$ en la que se calculan las longitudes de los caminos más cortos. Inicialmente se hace $A[i, j] = C[i, j]$ para toda $i \neq j$. Si no existe un arco que vaya de i a j , se supone que $C[i, j] = \infty$. Cada elemento de la diagonal se hace 0.

Después, se hacen n iteraciones en la matriz A . Al final de la k -ésima iteración, $A[i, j]$ tendrá por valor la longitud más pequeña de cualquier camino que vaya desde el vértice i hasta el vértice j y que no pase por un vértice con número mayor que k . Esto es, i y j , los vértices extremos del camino, pueden ser cualquier vértice, pero todo vértice intermedio debe ser menor o igual que k .

En la k -ésima iteración se aplica la siguiente fórmula para calcular A .

$$A_k[i, j] = \min \begin{cases} A_{k-1}[i, j] \\ A_{k-1}[i, k] + A_{k-1}[k, j] \end{cases}$$

El subíndice k denota el valor de la matriz A después de la k -ésima iteración; no indica la existencia de n matrices distintas. Pronto, se eliminarán esos subíndices. Esta fórmula tiene la interpretación simple de la figura 6.13.

Para obtener $A_k[i, j]$, se compara $A_{k-1}[i, j]$, el costo de ir de i a j sin pasar por k o cualquier otro vértice con numeración mayor, con $A_{k-1}[i, k] + A_{k-1}[k, j]$, el costo de ir primero de i a k y después de k a j , sin pasar a través de un vértice con número mayor que k . Si el paso por el vértice k produce un camino más económico que el de $A_{k-1}[i, j]$, se elige ese costo para $A_k[i, j]$.

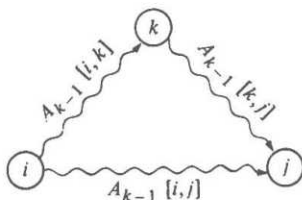


Fig. 6.13. Inclusión de k entre los vértices que van de i a j .

Ejemplo 6.8. Considérese el grafo dirigido ponderado que se muestra en la figura 6.14. En la figura 6.15 se muestran los valores iniciales de la matriz A , y después de tres iteraciones. $\square$

Como $A_k[i, k] = A_{k-1}[i, k]$ y $A_k[k, j] = A_{k-1}[k, j]$, ninguna entrada con cualquier subíndice igual a k cambia durante la k -ésima iteración. Por tanto, se puede realizar el cálculo sólo con una copia de la matriz A . En la figura 6.16 se muestra un programa para realizar este cálculo en matrices de $n \times n$.

Es evidente que el tiempo de ejecución de este programa es $O(n^3)$, ya que el programa está conformado por el triple ciclo anidado **for**. Para verificar que este progra-

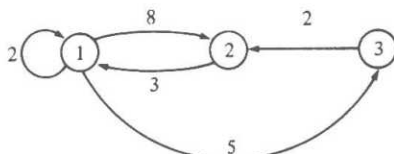


Fig. 6.14. Grafo dirigido ponderado.

ma funciona, es fácil demostrar por inducción sobre k que después de k recorridos por el triple ciclo **for**, $A[i, j]$ contiene la longitud del camino más corto desde el vértice i hasta el vértice j que no pasa a través de un vértice con número mayor que k .

	1	2	3		1	2	3
1	0	8	5	1	0	8	5
2	3	0	∞	2	3	0	8
3	∞	2	0	3	∞	2	0
	$A_0[i, j]$				$A_1[i, j]$		
	1	2	3		1	2	3
1	0	8	5	1	0	7	5
2	3	0	8	2	3	0	8
3	5	2	0	3	5	2	0
	$A_2[i, j]$				$A_3[i, j]$		

 Fig. 6.15. Valores de matrices A sucesivas.

```

procedure Floyd ( var  $A$ : array[1.. $n$ , 1.. $n$ ] of real;
     $C$ : array[1.. $n$ , 1.. $n$ ] of real );
{ Floyd calcula la matriz  $A$  de caminos más cortos dada la matriz de costos
  de arcos  $C$  }
var
     $i, j, k$ : integer;
begin
    for  $i$  := 1 to  $n$  do
        for  $j$  := 1 to  $n$  do
             $A[i, j]$  :=  $C[i, j]$ ;
    for  $i$  := 1 to  $n$  do
         $A[i, i]$  := 0;
    for  $k$  := 1 to  $n$  do
        for  $i$  := 1 to  $n$  do
            for  $j$  := 1 to  $n$  do
                if ( $A[i, k] + A[k, j]$ ) <  $A[i, j]$  then
                     $A[i, j]$  :=  $A[i, k] + A[k, j]$ 
end; { Floyd }
    
```

Fig. 6.16. Algoritmo de Floyd.

Comparación entre los algoritmos de Floyd y Dijkstra

Dado que la versión de *Dijkstra* con matriz de adyacencia puede encontrar los caminos más cortos desde un vértice en un tiempo $O(n^2)$, como el algoritmo de Floyd, también puede encontrar todos los caminos más cortos en un tiempo $O(n^3)$. El compilador, la máquina y los detalles de realización determinarán las constantes de proporcionalidad. La experimentación y medición son la forma más fácil de descubrir el mejor algoritmo para la aplicación en cuestión.

Si a , el número de aristas, es mucho menor que n^2 , aun con el factor constante relativamente bajo en el tiempo de ejecución $O(n^3)$ de *Floyd*, cabe esperar que la versión de *Dijkstra* con lista de adyacencia, tomando un tiempo $O(na \log n)$ para resolver el CMCP, sea superior, al menos para grafos grandes y poco densos.

Recuperación de los caminos

En muchos casos se desea imprimir el camino más económico entre dos vértices. Un modo de lograrlo es usando otra matriz P , donde $P[i, j]$ tiene el vértice k que permitió a *Floyd* encontrar el valor más pequeño de $A[i, j]$. Si $P[i, j] = 0$, el camino más corto de i a j es directo, siguiendo el arco entre ambos. La versión modificada de *Floyd* de la figura 6.17 almacena los vértices intermedios apropiados en P .

```

procedure más_corto ( var A: array[1..n, 1..n] of real;
    C: array[1..n, 1..n] of real; P: array[1..n, 1..n] of integer );
{ más_corto toma una matriz de costos de arcos C de  $n \times n$  y produce una matriz A
  de  $n \times n$  de longitudes de caminos más cortos y una matriz P de  $n \times n$ 
  que da un punto en la «mitad» de cada camino más corto }

var
    i, j, k: integer;
begin
    for i := 1 to n do
        for j := 1 to n do begin
            A[i, j] := C[i, j];
            P[i, j] := 0;
        end;
    for i := 1 to n do
        A[i, i] := 0;
    for k := 1 to n do
        for i := 1 to n do
            for j := 1 to n do
                if A[i, k] + A[k, j] < A[i, j] then begin
                    A[i, j] := A[i, k] + A[k, j];
                    P[i, j] := k;
                end
            end
        end;
    end; { más_corto }

```

Fig. 6.17. Programa para los caminos más cortos.

Para imprimir los vértices intermedios del camino más corto del vértice i hasta el vértice j , se invoca el procedimiento $\text{camino}(i, j)$ dado en la figura 6.18. Mientras que en una matriz arbitraria P , camino puede iterar infinitamente, si P viene del procedimiento más_corto , no es posible tener, por ejemplo, k en el camino más corto de i a j y también tener j en el camino más corto de i a k . Obsérvese cómo la suposición de pesos no negativos es crucial otra vez.

```

procedure camino ( i, j: integer );
  var
    k: integer;
  begin
    k := P[i, j];
    if k = 0 then
      return;
    camino(i, k);
    writeln(k);
    camino(k, j)
  end; { camino }

```

Fig. 6.18. Procedimiento para imprimir el camino más corto.

Ejemplo 6.9. La figura 6.19 muestra la matriz P final para el grafo dirigido de la figura 6.14. □

	1	2	3
1	0	3	0
2	0	0	1
3	2	0	0

P

Fig. 6.19. Matriz P para el grafo dirigido de la figura 6.14.

Cerradura transitiva

En algunos problemas podría ser interesante saber sólo si existe un camino de longitud igual o mayor que uno que vaya desde el vértice i al vértice j . El algoritmo de Floyd puede especializarse para este problema; el algoritmo resultante, que antecede al de Floyd, se conoce como algoritmo de Warshall.

Supóngase que la matriz de costo C es sólo la matriz de adyacencia para el grafo dirigido dado. Esto es, $C[i, j] = 1$ si hay un arco de i a j , y 0 si no lo hay. Se desea obtener la matriz A tal que $A[i, j] = 1$ si hay un camino de longitud igual o mayor que uno de i a j , y 0 en otro caso. A se conoce a menudo como *cerradura transitiva* de la matriz de adyacencia.

Ejemplo 6.10. La figura 6.20 muestra la cerradura transitiva para la matriz de adyacencia del grafo dirigido de la figura 6.14. □

	1	2	3
1	0	1	1
2	1	0	1
3	1	1	0

Fig. 6.20. Cerradura transitiva.

La cerradura transitiva puede obtenerse con un procedimiento similar a *Floyd* aplicando la siguiente fórmula en el k -ésimo paso en la matriz booleana A .

$$A_k[i, j] = A_{k-1}[i, j] \text{ o } (A_{k-1}[i, k] \text{ y } A_{k-1}[k, j])$$

Esta fórmula establece que hay un camino de i a j que no pasa por un vértice con número mayor que k si

1. ya existe un camino de i a j que no pasa por un vértice con número mayor que $k-1$, o si
2. hay un camino de i a k que no pasa por un vértice con número mayor que $k-1$, y un camino de k a j que no pasa por un vértice con número mayor que $k-1$.

Igual que antes, $A_k[i, k] = A_{k-1}[i, k]$ y $A_k[k, j] = A_{k-1}[k, j]$, así que se puede reutilizar el cálculo con sólo una copia de la matriz A . El programa en Pascal resultante, llamado *Warshall* por su descubridor, se muestra en la figura 6.21.

```

procedure Warshall ( var A: array[1..n, 1..n] of boolean;
                    C: array[1..n, 1..n] of boolean );
{ Warshall convierte a A en la cerradura transitiva de C }
var
    i, j, k: integer;
begin
    for i := 1 to n do
        for j := 1 to n do
            A[i, j] := C[i, j];
    for k := 1 to n do
        for i := 1 to n do
            for j := 1 to n do
                if A[i, j] = false then
                    A[i, j] := A[i, k] and A[k, j]
end; { Warshall }

```

Fig. 6.21. Algoritmo de Warshall para cerradura transitiva.

Un ejemplo: localización del centro de un grafo dirigido

Supóngase que se desea determinar el vértice más central de un grafo dirigido. Este problema puede resolverse fácilmente con el algoritmo de Floyd. Primero, se hace

más preciso el término «vértice más central». Sea v un vértice de un grafo dirigido $G = (V, A)$. La *excentricidad* de v es

$$\max_{w \in V} \{ \text{longitud mínima de un camino de } w \text{ a } v \}$$

El *centro* de G es un vértice de mínima excentricidad. Así, el centro de un grafo dirigido es un vértice más cercano al vértice más distante.

Ejemplo 6.11. Considérese el grafo dirigido ponderado de la figura 6.22.

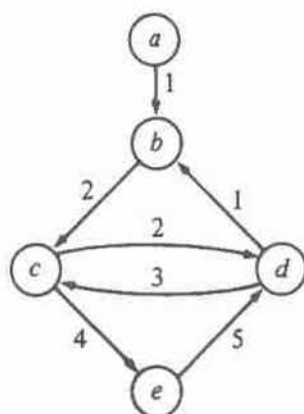


Fig. 6.22. Grafo dirigido ponderado.

Las excentricidades de los vértices son

vértice	excentricidad
a	∞
b	6
c	8
d	5
e	7

Por tanto, el centro es el vértice d . $\square$

Encontrar el centro de un grafo dirigido G es fácil. Supóngase que C es la matriz de costos para G .

1. Primero se aplica el procedimiento *Floyd* de la figura 6.16 a C para obtener la matriz A de los caminos más cortos entre todos los pares.
2. Se encuentra el costo máximo en cada columna i ; esto da la excentricidad del vértice i .
3. Se encuentra el vértice con excentricidad mínima; éste es el centro de G .

El tiempo de ejecución de este proceso está dominado por el primer paso, que lleva un tiempo $O(n^3)$. El paso (2) lleva $O(n^2)$ y el paso (3) lleva $O(n)$.

Ejemplo 6.12. La matriz de costo CMCP para la figura 6.22 se muestra en la figura 6.23. El valor máximo de cada columna se muestra a continuación.

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>
<i>a</i>	0	1	3	5	7
<i>b</i>	∞	0	2	4	6
<i>c</i>	∞	3	0	2	4
<i>d</i>	∞	1	3	0	7
<i>e</i>	∞	6	8	5	0
máx	∞	6	8	5	7

Fig. 6.23. Matriz de costo CMCP.

6.5 Recorridos en grafos dirigidos

Para resolver con eficiencia muchos problemas relacionados con grafos dirigidos, es necesario visitar los vértices y los arcos de manera sistemática. La búsqueda en profundidad, una generalización del recorrido en orden previo de un árbol, es una técnica importante para lograrlo, y puede servir como estructura para construir otros algoritmos eficientes. Las dos últimas secciones de este capítulo contienen varios algoritmos que usan esta búsqueda como base.

Supóngase que se tiene un grafo dirigido G en el cual todos los vértices están marcados en principio como *no visitados*. La búsqueda en profundidad trabaja seleccionando un vértice v de G como vértice de partida; v se marca como *visitado*. Después, se recorre cada vértice adyacente a v no visitado, usando recursivamente la búsqueda en profundidad. Una vez que se han visitado todos los vértices que se pueden alcanzar desde v , la búsqueda de v está completa. Si algunos vértices quedan sin visitar, se selecciona alguno de ellos como nuevo vértice de partida, y se repite este proceso hasta que todos los vértices de G se hayan visitado.

Esta técnica se conoce como búsqueda en profundidad porque continúa buscando en la dirección hacia adelante (más profunda) mientras sea posible. Por ejemplo, supóngase que x es el vértice visitado más recientemente. La búsqueda en profundidad selecciona algún arco no explorado $x \rightarrow y$ que parta de x . Si se ha visitado y , el procedimiento intenta continuar por otro arco que no se haya explorado y que parta de x . Si y no se ha visitado, entonces el procedimiento marca y como visitado e inicia una nueva búsqueda a partir de y . Después de completar la búsqueda de todos los caminos que parten de y , la búsqueda regresa a x , el vértice desde el cual se visitó y por primera vez. Se continúa el proceso de selección de arcos sin explorar que parten de x hasta que todos los arcos de x han sido explorados.

Puede usarse una lista de adyacencia $L[v]$ para representar los vértices adyacentes al vértice v , y un arreglo *marca* cuyos elementos son del tipo (*visitado*, *no_visi-*

tado), puede usarse para determinar si un vértice ya fue visitado antes. El procedimiento recursivo *bpf* se describe en la figura 6.24. Para usarlo en un grafo con n vértices, se asigna el valor inicial *marca* a *no_visitado*, y después se comienza la búsqueda en profundidad con cada vértice que aún permanezca sin visitar cuando llegue su turno, con

```

for  $v := 1$  to  $n$  do
     $marca[v] := no\_visitado$ ;
for  $v := 1$  to  $n$  do
    if  $marca[v] = no\_visitado$  then
        bpf( $v$ )

```

Obsérvese que la figura 6.24 es un modelo al cual se agregarán después otras acciones, al aplicar la búsqueda en profundidad. Lo único que hace el código de la figura 6.24 es actualizar el arreglo *marca*.

Análisis de la búsqueda en profundidad

Todas las llamadas a *bpf* en la búsqueda en profundidad de un grafo con a arcos y $n \leq a$ vértices lleva un tiempo $O(a)$. Para ver por qué, obsérvese que en ningún vértice se llama a *bpf* más de una vez, porque tan pronto como se llama a *bpf*(v) se hace *marca*[v] igual a *visitado* en la línea (1), y nunca se llama a *bpf* en un vértice que antes tenía su *marca* igual a *visitado*. Así, el tiempo total consumido en las líneas (2) y (3) recorriendo las listas de adyacencias es proporcional a la suma de las longitudes de dichas listas, esto es, $O(a)$. De esta forma, suponiendo que $n \leq a$, el tiempo total consumido por la búsqueda en profundidad de un grafo completo es $O(a)$, lo cual es, hasta un factor constante, el tiempo necesario simplemente para recorrer cada arco.

```

procedure bpf (  $v$ : vértice );
    var
         $w$ : vértice;
    begin
(1)       $marca[v] := visitado$ ;
(2)      for cada vértice  $w$  en  $L[v]$  do
(3)          if  $marca[w] = no\_visitado$  then
(4)              bpf( $w$ )
    end; | bpf |

```

Fig. 6.24. Búsqueda en profundidad.

Ejemplo 6.13. Supóngase que el procedimiento *bpf*(v) se aplica al grafo dirigido de la figura 6.25 con $v = A$. El algoritmo marca A como visitado y selecciona el vértice B en las lista de adyacencia del vértice A . Puesto que B no se ha visitado, la búsqueda continúa llamando a *bpf*(B). El algoritmo marca ahora B como visitado y selec-

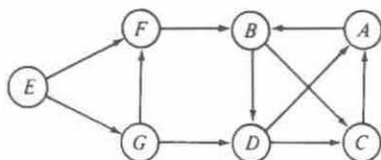


Fig. 6.25. Grafo dirigido.

ciona el primer vértice en la lista de adyacencia del vértice B . Dependiendo del orden de los vértices en la lista de adyacencia de B , la búsqueda seguirá con C o D .

Suponiendo que C aparece antes que D , se invoca a $bpf(C)$. El vértice A está en la lista de adyacencia de C . Sin embargo, en este momento ya se ha visitado A así que la búsqueda queda en C . Como ya se han visitado todos los vértices de la lista de adyacencia en C , la búsqueda regresa a B , desde donde la búsqueda prosigue a D . Los vértices A y C en la lista de adyacencia de D ya fueron visitados, por lo que la búsqueda regresa a B y después a A .

En este punto, la llamada original a $bpf(A)$ ha terminado. Sin embargo, el grafo dirigido no ha sido recorrido en su totalidad; los vértices E , F y G están sin visitar. Para completar la búsqueda, se puede llamar a $bpf(E)$.

Bosque abarcador en profundidad

Durante un recorrido en profundidad de un grafo dirigido, cuando se recorren ciertos arcos, llevan a vértices sin visitar. Los arcos que llevan a vértices nuevos se conocen como *arcos de árbol* y forman un *bosque abarcador en profundidad* para el grafo dirigido dado. Los arcos continuos de la figura 6.26 forman el bosque abarcador en profundidad del grafo dirigido de la figura 6.25. Obsérvese que los arcos de árbol deben formar realmente un bosque, ya que un vértice no puede estar sin visitar cuando se recorren dos arcos diferentes que llevan a él.

Además de los arcos de árbol, existen otros tres tipos de arcos definidos por una búsqueda en profundidad de un grafo dirigido, que se conocen como arcos de retroceso, arcos de avance y arcos cruzados. Un arco como $C \rightarrow A$ se denomina *arco de retroceso*, porque va de un vértice a uno de sus antecesores en el bosque abarcador. Obsérvese que un arco que va de un vértice hacia sí mismo, es un arco de retroceso. Un arco no abarcador que va de un vértice a un descendiente propio se llama *arco de avance*. En la figura 6.25 no hay arcos de este tipo.

Los arcos como $D \rightarrow C$ o $G \rightarrow D$, que van de un vértice a otro que no es antecesor ni descendiente, se conocen como *arcos cruzados*. Obsérvese que todos los arcos cruzados de la figura 6.26 van de derecha a izquierda, en el supuesto de que se agregan hijos al árbol en el orden en que fueron visitados, de izquierda a derecha, y que se agregan árboles nuevos al bosque de izquierda a derecha. Este patrón no es accidental. Si el arco $G \rightarrow D$ hubiera sido $D \rightarrow G$, entonces no se hubiera visitado G durante la búsqueda en D , y al encontrar el arco $D \rightarrow G$, el vértice G se haría descendiente de D , y $D \rightarrow G$ se convertiría en un arco de árbol.

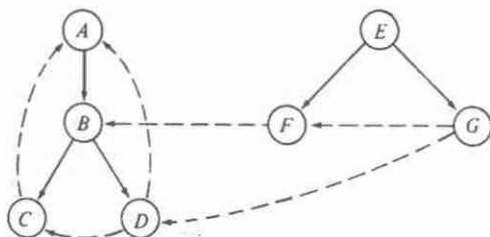


Fig. 6.26. Bosque abarcador en profundidad para la figura 6.25.

¿Cómo distinguir entre los cuatro tipos de arcos? Es obvio que los arcos de árbol son especiales, pues llevan a vértices sin visitar durante la búsqueda en profundidad. Supóngase que se numeran los vértices de un grafo dirigido de acuerdo con el orden en que se marcaron los visitados durante la búsqueda en profundidad. Esto es, se puede asignar a un arreglo.

```

númerop[v] := cont;
cont := cont + 1;
    
```

después de la línea (1) de la figura 6.24. A esto se le llama *numeración en profundidad* de un grafo dirigido; obsérvese que la numeración en profundidad generaliza la numeración en orden previo introducida en la sección 3.1.

La búsqueda en profundidad asigna a todos los descendientes de un vértice v , números mayores o iguales al número asignado a v . De hecho, w es un descendiente de v si, y sólo si, $númerop(v) \leq númerop(w) \leq númerop(v) + \text{el número de descendientes de } v$. Así, los arcos de avance van de los vértices de baja numeración a los de alta numeración y los arcos de retroceso van de los vértices de alta numeración a los de baja numeración.

Todos los arcos cruzados van de los vértices de alta numeración a los de baja numeración. Para ver esto, supóngase que $x \rightarrow y$ es un arco y $númerop(x) \leq númerop(y)$. Así, x se visita antes que y . Todo vértice visitado entre su invocación por primera vez a $bpf(x)$ y el momento en que $bpf(x)$ termina, se convierte en descendiente de x en el bosque abarcador en profundidad. Si y permanece sin visitar cuando se explora el arco $x \rightarrow y$, $x \rightarrow y$ se vuelve un arco de árbol. De otra forma, $x \rightarrow y$ es un arco de avance. Así, $x \rightarrow y$ no puede ser un arco cruzado con $númerop(x) \leq númerop(y)$.

En las dos secciones siguientes se analiza la forma de usar la búsqueda en profundidad para la solución de varios problemas de grafos.

6.6 Grafos dirigidos acíclicos

Un *grafo dirigido acíclico*, o *gda*, es un grafo dirigido sin ciclos. Cuantificados en función de las relaciones que representan, los gda son más generales que los árboles,

pero menos que los grafos dirigidos arbitrarios. La figura 6.27 muestra un ejemplo de un árbol, un gda, y un grafo dirigido con un ciclo.

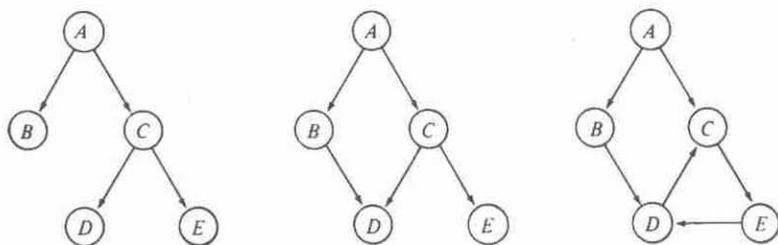


Fig. 6.27. Tres grafos dirigidos.

Entre otras cosas, los gda son útiles para la representación de la estructura sintáctica de expresiones aritméticas con subexpresiones comunes. Por ejemplo, la figura 6.28 muestra un gda para la expresión

$$((a + b) * c + ((a + b) + e) * (e + f)) * ((a + b) * c)$$

Los términos $a + b$ y $(a + b) * c$ son subexpresiones comunes compartidas que se representan con vértices con más de un arco entrante.

Los gda son útiles también para la representación de órdenes parciales. Un orden parcial R en un conjunto S es una relación binaria tal que

1. para toda a en S , $a R a$ es falsa (R es irreflexivo), y
2. para toda a, b, c en S , si $a R b$ y $b R c$, entonces $a R c$ (R es transitivo).

Dos ejemplos naturales de órdenes parciales son la relación «menor que» ($<$) en enteros, y la relación de inclusión propia en conjuntos ($\subset$).

Ejemplo 6.14. Sea $S = \{1, 2, 3\}$ y sea $P(S)$ el conjunto exponencial de S , esto es, el conjunto de todos los subconjuntos de S . $P(S) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$. $\subset$ es un orden parcial en $P(S)$. Ciertamente, $A \subset A$ es falso para cualquier conjunto A (irreflexividad), y si $A \subset B$ y $B \subset C$, entonces $A \subset C$ (transitividad). $\square$

Los gda pueden usarse para reflejar gráficamente órdenes parciales. Para empezar, se puede considerar una relación R como un conjunto de pares (arcos) tales que (a, b) está en el conjunto si, y sólo si, $a R b$ es cierto. Si R es un orden parcial en el conjunto S , entonces el grafo dirigido $G = (S, R)$ es un gda. Del mismo modo, supóngase que $G = (S, R)$ es un gda y R^* es la relación definida por $a R^* b$ si, y sólo si, existe un camino de longitud uno o más que va de a a b . (R^* es la cerradura transitiva de la relación R .) Entonces, R^* es un orden parcial en S .

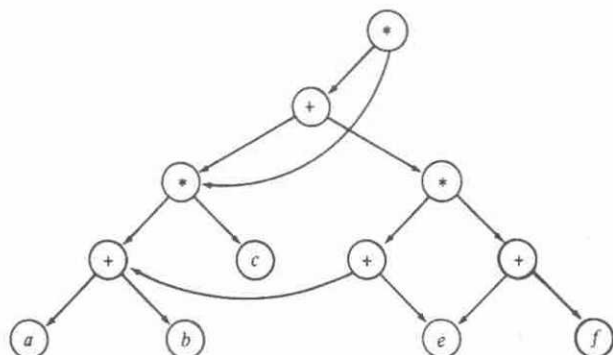


Fig. 6.28. Gda para expresiones aritméticas.

Ejemplo 6.15. La figura 6.29 muestra un gda $(P(S), R)$, donde $S = \{1, 2, 3\}$. La relación R^+ es la inclusión propia en el conjunto exponencial $P(S)$. $\square$

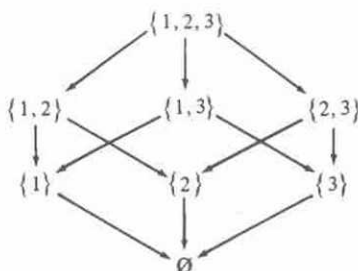


Fig. 6.29. Gda para inclusiones propias.

Prueba de aciclicidad

Se tiene un grafo dirigido $G = (V, A)$, para determinar si G es acíclico, esto es, si G no contiene ciclos. La búsqueda en profundidad puede usarse para responder a esta pregunta. Si se encuentra un arco de retroceso durante la búsqueda en profundidad de G , el grafo tiene un ciclo. Si, al contrario, un grafo dirigido tiene un ciclo, entonces siempre habrá un arco de retroceso en la búsqueda en profundidad del grafo.

Para ver este hecho, supóngase que G es cíclico. Si se efectúa una búsqueda en profundidad en G , habrá un vértice v que tenga el número de búsqueda en profundidad menor en un ciclo. Considérese un arco $u \rightarrow v$ en algún ciclo que contenga

a v . Ya que u está en el ciclo, debe ser un descendiente de v en el bosque abarcador en profundidad. Así, $u \rightarrow v$ no puede ser un arco cruzado. Puesto que el número en profundidad de u es mayor que el de v , $u \rightarrow v$ no puede ser un arco de árbol ni un arco de avance. Así que $u \rightarrow v$ debe ser un arco de retroceso, como se ilustra en la figura 6.30.



Fig. 6.30. Todo ciclo contiene un arco de retroceso.

Clasificación topológica

Un proyecto grande suele dividirse en una colección de tareas más pequeñas, algunas de las cuales se han de realizar en ciertos órdenes específicos, de modo que se pueda culminar el proyecto total. Por ejemplo, una carrera universitaria puede contener cursos que requieran otros como prerequisites. Los gda pueden emplearse para modelar de manera natural estas situaciones. Por ejemplo, podría tenerse un arco del curso C al curso D si C fuera un prerequisite de D .

Ejemplo 6.16. La figura 6.31 muestra un gda con la estructura de prerequisites de cinco cursos. El curso C_3 , por ejemplo, requiere los cursos C_1 y C_2 como prerequisites. □

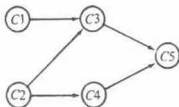


Fig. 6.31. Gda de prerequisites.

La *clasificación topológica* es un proceso de asignación de un orden lineal a los vértices de un gda tal que si existe un arco del vértice i al vértice j , i aparece antes que j en el ordenamiento lineal. Por ejemplo, $C1, C2, C3, C4, C5$ es una clasificación topológica del gda de la figura 6.31. Al tomar los cursos en esta secuencia, se puede satisfacer la estructura de prerrequisitos dada en la figura.

La clasificación topológica puede efectuarse con facilidad si se agrega una instrucción de impresión después de la línea (4) al procedimiento de búsqueda en profundidad de la figura 6.24:

```

procedure clasificación_topológica( $v$ : vértice);
  {imprime los vértices accesibles desde  $v$  en orden topológico invertido}
  var
     $w$ : vértice;
  begin
     $\text{marca}[v] := \text{visitado}$ ;
    for cada vértice  $w$  en  $L[v]$  do
      if  $\text{marca}[w] = \text{no\_visitado}$  then
        clasificación_topológica( $w$ );
       $\text{writeln}(v)$ 
    end; {clasificación_topológica}

```

Cuando *clasificación_topológica* termina de buscar en todos los vértices adyacentes a un vértice dado x , imprime x . El efecto de llamar a *clasificación_topológica*(v) es imprimir en orden topológico inverso todos los vértices de un gda accesibles desde v por medio de un camino en el gda.

Esta técnica funciona porque no existen arcos de retroceso en un gda. Considérese lo que sucede cuando la búsqueda en profundidad deja un vértice x por última vez. Los únicos arcos que emanan de v son arcos de árbol, de avance y cruzados. Pero todos esos arcos están dirigidos hacia vértices que ya se han visitado completamente y que, por tanto, preceden a x en el orden que se están construyendo.

6.7 Componentes fuertes

Un componente fuertemente conexo de un grafo dirigido es un conjunto maximal de vértices en el cual existe un camino que va desde cualquier vértice del conjunto hasta cualquier otro vértice también del conjunto. La búsqueda en profundidad puede usarse para determinar con eficiencia los componentes fuertemente conexos de un grafo dirigido.

Sea $G = (V, A)$ un grafo dirigido; se puede dividir V en clases de equivalencia V_i , $1 \leq i \leq r$, tales que los vértices v y w son equivalentes si, y sólo si, existe un camino de v a w y otro de w a v . Sea A_i , $1 \leq i \leq r$, el conjunto de los arcos con cabeza y cola en V_i . Los grafos $G_i = (V_i, A_i)$ se denominan *componentes fuertemente conexos* (o sólo *componentes fuertes*) de G . Un grafo dirigido con sólo un componente fuerte, se dice que está *fuertemente conexo*.

Ejemplo 6.17. La figura 6.32 ilustra un grafo dirigido con los dos componentes fuertes mostrados en la figura 6.33. $\square$

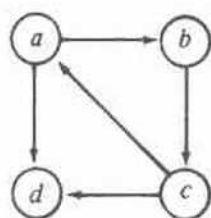


Fig. 6.32. Grafo dirigido.

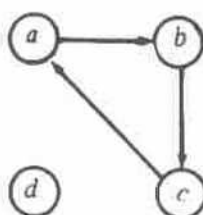


Fig. 6.33. Componentes fuertes del grafo de la figura 6.32.

Obsérvese que todo vértice de un grafo dirigido G está en algún componente fuerte, pero que ciertos arcos pueden no estarlo. Tales arcos, llamados arcos de *cruce de componentes*, van de un vértice de un componente a un vértice de otro. Se pueden representar las interconexiones entre los componentes construyendo un *grafo reducido* de G , cuyos vértices son los componentes fuertemente conexos de G . Hay un arco de un vértice C a un vértice diferente C' de este tipo de grafos, si existe un arco en G que vaya de algún vértice del componente C a algún otro del componente C' . El grafo reducido siempre es un gda, porque si existiera algún ciclo, todos los componentes del *ciclo* serían en realidad un solo componente fuerte, lo cual significaría que no se calcularon en forma adecuada los componentes fuertes. La figura 6.34 muestra el grafo reducido del grafo dirigido de la figura 6.32.



Fig. 6.34. Grafo reducido.

Ahora se presenta un algoritmo para encontrar los componentes fuertemente conexos de un grafo dirigido G dado.

1. Efectúese una búsqueda en profundidad de G y numérense los vértices en el orden de terminación de las llamadas recursivas; esto es, asígnese un número al vértice v después de la línea (4) de la figura 6.24.
2. Constrúyase un grafo dirigido nuevo G_r , invirtiendo las direcciones de todos los arcos de G .
3. Realícese una búsqueda en profundidad en G_r partiendo del vértice con numeración más alta de acuerdo con la numeración asignada en el paso (1). Si la búsqueda en profundidad no llega a todos los vértices, iníciase la siguiente búsqueda a partir del vértice restante con numeración más alta.
4. Cada árbol del bosque abarcador resultante es un componente fuertemente conexo de G .

Ejemplo 6.18. Se aplica este algoritmo al grafo dirigido de la figura 6.32, partiendo de a y continuando primero hasta b . Después del paso (1) se numeran los vértices como se muestra en la figura 6.35. Al invertir la dirección de los arcos, se obtiene el grafo G_r de la figura 6.36.

Cuando se realiza la búsqueda en profundidad en G_r surge el bosque abarcador en profundidad de la figura 6.37. Se comienza con a como raíz, porque a tiene el número más alto. Desde a sólo se alcanza c y después b . El siguiente árbol tiene raíz d , ya que es el siguiente (y único) vértice restante con numeración más alta. Cada árbol del bosque forma un componente fuertemente conexo del grafo dirigido original. $\square$

Se ha pretendido que los vértices de un componente fuertemente conexo se correspondan con los vértices de un árbol del bosque abarcador de la segunda búsqueda en profundidad. Para ver por qué, obsérvese que si v y w son vértices del mismo componente fuertemente conexo, existen caminos en G desde v hasta w y desde w hasta v . Así, existen también caminos desde v hasta w y desde w hasta v en G_r .

Supóngase que en la búsqueda en profundidad de G_r se inicia la búsqueda en alguna raíz x y se llega hasta v o w . Como v y w se alcanzan uno al otro, ambos terminarán formando parte del árbol abarcador con raíz x .

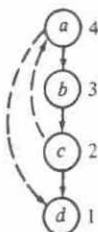
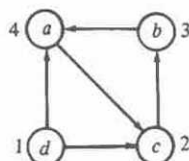
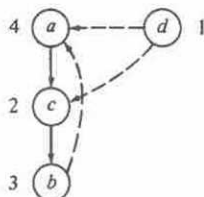


Fig. 6.35. Después del paso 1.

Fig. 6.36. G_r .Fig. 6.37. Bosque abarcador en profundidad para G_r .

Ahora, supóngase que v y w están en el mismo árbol abarcador del bosque abarcador en profundidad de G_r . Se debe demostrar que v y w están en el mismo componente fuertemente conexo. Sea x la raíz del árbol abarcador que contiene v y w . Puesto que v es descendiente de x , existe un camino en G_r que va de x a v . Así, existe un camino en G de v a x .

En la construcción del bosque abarcador en profundidad de G_r , el vértice v quedó sin visitarse cuando se inició la búsqueda en x . De aquí que x tiene un número mayor que v , por lo que en la búsqueda en profundidad de G , la llamada recursiva en v terminó antes que la llamada recursiva en x . Pero en la búsqueda en profundidad de G , la búsqueda en v no pudo haberse iniciado antes que la de x , ya que el camino en G de v a x implicaría que la búsqueda en x empezaría y terminaría antes de terminar la búsqueda en v .

Se concluye que en la búsqueda de G , v se visita durante la búsqueda de x , por lo que, v es descendiente de x en el primer bosque abarcador en profundidad de G . Así, existe un camino de x a v en G . Por tanto, x y v están en el mismo componente fuertemente conexo. Un razonamiento idéntico muestra que x y w están en el mismo componente fuertemente conexo y, por tanto, v y w también lo están, como muestran los caminos que van de v a x a w , y de w a x a v .

Ejercicios

6.1 Representése el grafo dirigido de la figura 6.38

- por medio de una matriz de adyacencia dando los costos de los arcos, y
- por medio de una lista enlazada de adyacencia con indicación de los costos de los arcos.

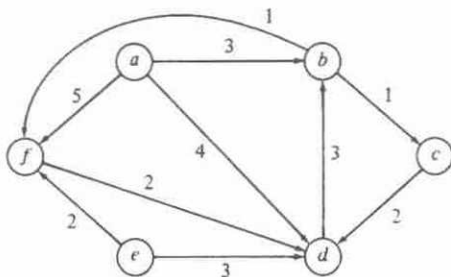


Fig. 6.38. Grafo dirigido con costos de los arcos.

- 6.2 Describese un modelo matemático para el siguiente problema de horarios. Dadas las tareas $T_1, T_2, \dots, T_m$, que requieren tiempos $t_1, t_2, \dots, t_m$ para ejecutarse, y un conjunto de restricciones, cada una de la forma « T_i debe terminar antes del inicio de T_j », encuéntrase el tiempo mínimo necesario para ejecutar todas las tareas.
- 6.3 Realícense las operaciones PRIMERO, SIGUIENTE y VERTICE para grafos dirigidos representados por
- matrices de adyacencia,
 - listas enlazadas de adyacencias, y
 - listas de adyacencias como la representada en la figura 6.5.
- 6.4 En el grafo dirigido de la figura 6.38,
- empléese el algoritmo *Dijkstra* para encontrar los caminos más cortos que van del vértice a a los otros vértices.
 - utilícese el algoritmo *Floyd* para encontrar las distancias más cortas entre todos los pares de puntos. Constrúyase también la matriz P que permita recuperar los caminos más cortos.
- 6.5 Escribese un programa completo para el algoritmo de *Dijkstra* usando árboles parcialmente ordenados como colas de prioridad y listas enlazadas de adyacencias.
- *6.6 Demuéstrase que el programa *Dijkstra* no funciona bien si los arcos tienen costos negativos.
- **6.7 Muéstrase que el programa *Floyd* funciona si alguno de los arcos, pero ningún ciclo, tienen costo negativo.
- 6.8 Suponiendo que el orden de los vértices es $a, b, \dots, f$ en la figura 6.38, constrúyase un bosque abarcador en profundidad; indíquese los arcos de árbol, de retroceso, de avance y cruzados, y la numeración en profundidad de los vértices.

- *6.9 Supóngase que se tiene un bosque abarcador en profundidad, y se lista en orden posterior cada uno de los árboles abarcadores (los árboles que están formados por aristas abarcadoras), de izquierda a derecha. Demuéstrese que este orden es el mismo en el que terminaron las llamadas a *bpf* cuando se construyó el bosque abarcador.
- 6.10 Una raíz de un gda es un vértice r tal que todo vértice del gda puede alcanzarse por un camino dirigido desde r . Escribese un programa para determinar si un gda posee raíz.
- *6.11 Considérese un gda con a arcos y con dos vértices distintos s y t . Constrúyase un algoritmo $O(a)$ para encontrar el conjunto maximal de caminos disjuntos de s a t . Por maximal se entiende que ya no se pueden añadir caminos adicionales, pero eso no significa que sea el tamaño más grande para ese conjunto.
- 6.12 Constrúyase un algoritmo para convertir un árbol de expresiones con los operadores $+$ y $*$ en un gda al compartir subexpresiones comunes. ¿Cuál es la complejidad de tiempo de ese algoritmo?
- 6.13 Constrúyase un algoritmo para la evaluación de expresiones aritméticas representadas con un gda.
- 6.14 Escribese un programa para encontrar el camino más largo en un gda. ¿Cuál es la complejidad de tiempo de este programa?
- 6.15 Encuéntrense los componentes fuertes de la figura 6.38.
- *6.16 Pruébese que el grafo reducido de los componentes fuertes de la sección 6.7 debe ser un gda.
- 6.17 Dibújese el primer bosque abarcador, el grafo invertido y el segundo bosque abarcador que se obtiene al aplicar el algoritmo de componentes fuertes al grafo dirigido de la figura 6.38.
- 6.18 Obténgase el algoritmo de componentes fuertes analizado en la sección 6.7.
- *6.19 Muéstrase que el algoritmo de componentes fuertes requiere un tiempo $O(a)$ en un grafo dirigido de a arcos y n vértices, suponiendo que $n \leq a$.
- *6.20 Escribese un programa que tome como entrada un grafo dirigido y dos de sus vértices. El programa debe imprimir todos los caminos simples que vayan de un vértice al otro. ¿Cuál es la complejidad de tiempo de este programa?
- *6.21 Una *reducción transitiva* de un grafo dirigido $G = (V, A)$ es cualquier grafo G' con los mismos vértices pero con la menor cantidad de arcos posible, de modo que el cierre transitivo G' es el mismo que el de G . Demuéstrese que si G es un gda, la reducción transitiva de G es única.

- *6.22 Escribese un programa para obtener la reducción transitiva de un grafo dirigido. ¿Cuál es la complejidad de tiempo de este programa?
- *6.23 $G' = (V, A')$ se conoce como el *grafo dirigido equivalente minimal* de un grafo dirigido $G = (V, A)$, si A' es el subconjunto más pequeño de A y la cerradura transitiva de G y G' es el mismo. Demuéstrese que si G es acíclico, sólo hay un grafo dirigido equivalente minimal, es decir, la reducción transitiva.
- *6.24 Escribese un programa para encontrar un grafo dirigido equivalente minimal para un grafo dirigido dado. ¿Cuál es la complejidad de tiempo de ese programa?
- *6.25 Escribese un programa para encontrar el camino simple más largo de un vértice dado de un grafo dirigido. ¿Cuál es la complejidad de tiempo del programa?

Notas bibliográficas

Berge [1985] y Harary [1969] son dos buenas fuentes de material suplementario sobre teoría de grafos. Algunos libros que tratan algoritmos sobre grafos son Deo [1975], Even [1980] y Tarjan [1983].

El algoritmo para caminos más cortos con un solo origen de la sección 6.3 se debe a Dijkstra [1959]. El algoritmo de los caminos más cortos entre todos los pares es de Floyd [1962] y el de cerradura transitiva es de Warshall [1962]. Johnson [1977] analiza algoritmos eficientes para encontrar caminos más cortos en grafos dispersos. Knuth [1968] contiene material adicional sobre clasificación topológica.

El algoritmo de componentes fuertes de la sección 6.7 es similar al sugerido por R. Kosaraju en 1978 (sin publicar), y al publicado por Sharir [1981]. Tarjan [1972] contiene otro algoritmo de componentes fuertes que sólo necesita un recorrido con búsqueda en profundidad.

Coffman [1976] contiene muchos ejemplos de cómo se pueden usar los grafos dirigidos para los problemas de modelado de horarios, como en el ejercicio 6.2. Aho, Garey y Ullman [1972] muestran que la reducción transitiva de un gda es única, y que el cálculo de la reducción transitiva de un grafo dirigido es, computacionalmente, equivalente al cálculo de la cerradura transitiva (Ejercicios 6.21 y 6.22). La obtención del grafo dirigido equivalente minimal (Ejercicios 6.23 y 6.24), por otro lado, parece ser mucho más difícil desde el punto de vista computacional; este problema es NP-completo [Sahni (1974)].

7

Grafos no dirigidos

Un grafo no dirigido $G = (V, A)$ consta de un conjunto finito de vértices V y de un conjunto de aristas A . Se diferencia de un grafo dirigido en que cada arista en A es un par no ordenado de vértices $\dagger$. Si (v, w) es una arista no dirigida, entonces $(v, w) = (w, v)$. De ahora en adelante se hará referencia a los grafos no dirigidos tan sólo como grafos.

Los grafos se emplean en distintas disciplinas para modelar relaciones simétricas entre objetos. Los objetos se representan por los vértices del grafo, y dos objetos están conectados por una arista si están relacionados entre sí. En este capítulo se presentan varias estructuras de datos que pueden usarse para representar grafos, y los algoritmos para tres problemas comunes que se relacionan con grafos no dirigidos: construcción de árboles abarcadores minimales, componentes biconexos y comparaciones maximales.

7.1 Definiciones

Buena parte de la terminología para grafos dirigidos es aplicable también a los no dirigidos. Por ejemplo, los vértices v y w son *adyacentes* si (v, w) es una arista [o, en forma equivalente, si (w, v) lo es]. Se dice que la arista (v, w) es *incidente* sobre los vértices v y w .

Un *camino* es una secuencia de vértices $v_1, v_2, \dots, v_n$ tal que (v_i, v_{i+1}) es una arista para $1 \leq i < n$. Un camino es *simple* si todos sus vértices son distintos, con excepción de v_1 y v_n , que pueden ser el mismo. La longitud del camino es $n - 1$, el número de aristas a lo largo del camino. Se dice que el camino $v_1, v_2, \dots, v_n$ *conecta* v_1 y v_n . Un grafo es *conexo* si todos sus pares de vértices están conectados.

Sea $G = (V, A)$ un grafo con conjunto de vértices V y conjunto de aristas A . Un *subgrafo* de G es un grafo $G' = (V', A')$ donde

1. V' es un subconjunto de V .
2. A' consta de las aristas (v, w) en A tales que v y w están en V' .

Si A' consta de todas las aristas (v, w) en A , tal que v y w están en V' , entonces G' se conoce como un *subgrafo inducido* de G .

$\dagger$ A menos que se especifique lo contrario, aquí se supondrá que una arista siempre es un par de vértices distintos.

Ejemplo 7.1. En la figura 7.1(a) se observa un grafo $G = (V, A)$ con $V = \{a, b, c, d\}$ y $A = \{(a, b), (a, d), (b, c), (b, d), (c, d)\}$, y en la figura 7.1(b), uno de sus subgrafos inducidos, definido por el conjunto de vértices $\{a, b, c\}$ y todas las aristas de la figura 7.1(a) que no inciden sobre el vértice d . $\square$

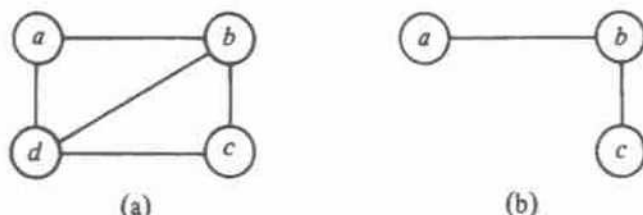


Fig. 7.1. Grafo con uno de sus subgrafos.

Un *componente conexo* de un grafo G es un subgrafo conexo inducido maximal, esto es, un subgrafo conexo inducido que por sí mismo no es un subgrafo propio de ningún otro subgrafo conexo de G .

Ejemplo 7.2. La figura 7.1 es un grafo conexo que tiene sólo un componente conexo, y que es él mismo. La figura 7.2 es un grafo con dos componentes conexos. $\square$

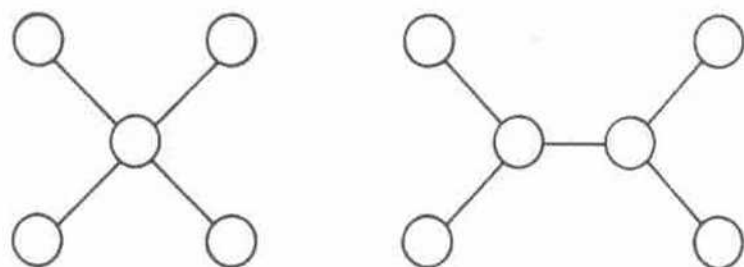


Fig. 7.2. Grafo no conexo.

Un *ciclo* (simple) de un grafo es un camino (simple) de longitud mayor o igual a tres, que conecta un vértice consigo mismo. No se consideran ciclos los caminos de la forma v (camino de longitud 0), v, v (camino de longitud 1), o v, w, v (camino de longitud 2). Un grafo es *cíclico* si contiene por lo menos un ciclo. Un grafo conexo acíclico algunas veces se conoce como *árbol libre*. La figura 7.2 muestra un grafo que consta de dos componentes conexos, cada uno de los cuales es un árbol libre. Un árbol libre puede convertirse en ordinario si se elige cualquier vértice deseado como raíz y se orienta cada arista desde ella.

Los árboles libres tienen dos propiedades importantes que se usarán en la siguiente sección.

1. Todo árbol libre con $n \geq 1$ vértices contiene exactamente $n - 1$ aristas.
2. Si se agrega cualquier arista a un árbol libre, resulta un ciclo.

Se puede probar (1) por inducción en n , o en forma equivalente, con un argumento basado en el "contraejemplo más pequeño". Supóngase que $G = (V, A)$ es un contraejemplo de (1) con un mínimo de vértices n , por ejemplo n no puede valer uno, porque el único árbol libre con un vértice tiene cero aristas, y (1) se satisface. Por tanto, n debe ser mayor que uno.

Ahora se pretende que en el árbol libre exista algún vértice con exactamente una arista incidente. En la demostración, ningún vértice puede tener cero aristas incidentes, o G no sería conexo. Supóngase que todo vértice tiene por lo menos dos aristas incidentes. Después, pártase de algún vértice v_1 , y sígase cualquier arista desde v_1 . En cada paso, se abandona un vértice por una arista diferente a la que se utilizó para llegar, formando un camino $v_1, v_2, v_3, \dots$.

Dado que sólo se tiene un número finito de vértices en V , no es posible que todos los vértices en el camino sean diferentes; en un momento dado, se encuentra $v_i = v_j$ para alguna $i < j$. No se puede tener $i = j - 1$, porque no hay ciclos de un vértice a sí mismo, y tampoco $i = j - 2$, ya que se llegaría y se abandonaría el vértice v_{i+1} por la misma arista. Así, $i \leq j - 3$, y se tiene un ciclo $v_i, v_{i+1}, \dots, v_j = v_i$, con lo que se contradice la hipótesis de que G no tiene vértices con sólo una arista incidente y, por tanto, se concluye que existe tal vértice v con arista (v, w) .

Ahora, considérese el grafo G' formado al eliminar el vértice v y la arista (v, w) de G . G' no puede contradecir (1), porque si lo hiciera podría ser un contraejemplo más pequeño que G . Por tanto, G' tiene $n - 1$ vértices y $n - 2$ aristas. Pero G tiene un vértice y una arista más que G' , es decir, tiene $n - 1$ aristas, probando que G satisface realmente (1). Como no hay un contraejemplo más pequeño para (1), se concluye que no existe ese contraejemplo, y (1) es cierto.

Ahora, es posible probar con facilidad la proposición (2) de que la adición de una arista a un árbol libre forma un ciclo. De no ser así, el resultado de agregar la arista a un árbol libre de n vértices sería un grafo con n vértices y n aristas. Este grafo aún sería conexo, y se ha supuesto que agregando la arista quedaría un grafo acíclico. Con esto, se tendría un árbol libre cuyas cantidades de vértices y de aristas no satisfarían la condición (1).

Métodos de representación

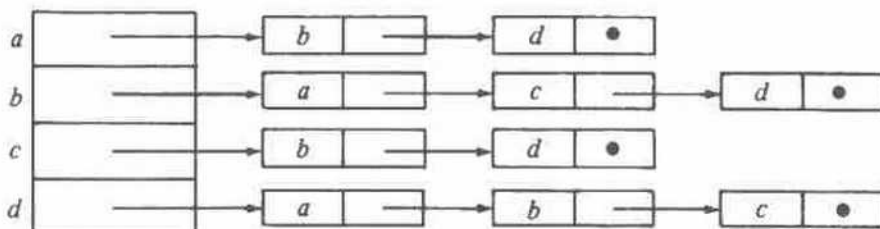
Los métodos de representación de grafos dirigidos se pueden emplear también para representar los no dirigidos. Una arista no dirigida entre v y w se representa simplemente con dos aristas dirigidas, una de v a w , y otra de w a v .

Ejemplo 7.3. Las representaciones con matriz y lista de adyacencia para el grafo de la figura 7.1(a) se muestran en la figura 7.3. $\square$

Es notorio que la matriz de adyacencia para un grafo es simétrica. En la representación con lista de adyacencia, si (i, j) es una arista, el vértice j estará en la lista del vértice i y el vértice i estará en la lista del vértice j .

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	0	1	0	1
<i>b</i>	1	0	1	1
<i>c</i>	0	1	0	1
<i>d</i>	1	1	1	0

(a) Matriz de adyacencia



(b) Lista de adyacencia

Fig. 7.3. Representaciones.

7.2 Árboles abarcadores de costo mínimo

Supóngase que $G = (V, A)$ es un grafo conexo en donde cada arista (u, v) de A tiene un costo asociado $c(u, v)$. Un *árbol abarcador* para G es un árbol libre que conecta todos los vértices de V ; su *costo* es la suma de los costos de las aristas del árbol. En esta sección se muestra cómo obtener el árbol abarcador de costo mínimo para G .

Ejemplo 7.4. La figura 7.4 muestra un grafo ponderado y su árbol abarcador de costo mínimo. □

Una aplicación típica de los árboles abarcadores de costo mínimo tiene lugar en el diseño de redes de comunicación. Los vértices del grafo representan ciudades, y las aristas, las posibles líneas de comunicación entre ellas. El costo asociado a una arista representa el costo de seleccionar esa línea para la red. Un árbol abarcador de costo mínimo representa una red que comunica todas las ciudades a un costo minimal.

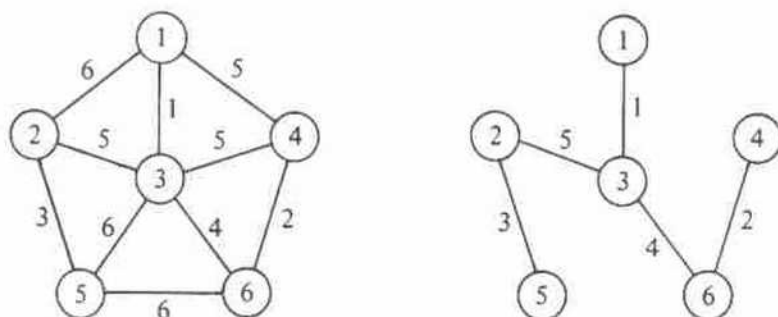


Fig. 7.4. Grafo y árbol abarcador.

La propiedad AAM (Árbol Abarcador de costo Mínimo)

Hay distintas formas de construir un árbol abarcador de costo mínimo. Muchos de esos métodos utilizan la siguiente propiedad de los árboles abarcadores de costo mínimo, que se denomina *propiedad AAM*. Sea $G = (V, A)$ un grafo conexo con una función de costo definida en las aristas. Sea U algún subconjunto propio del conjunto de vértices V . Si (u, v) es una arista de costo mínimo tal que $u \in U$ y $v \in V - U$, existe un árbol abarcador de costo mínimo que incluye (u, v) entre sus aristas.

La demostración de que todo árbol abarcador de costo mínimo satisface la propiedad AAM no es muy difícil. Supóngase, por el contrario, que no existe el árbol abarcador de costo mínimo para G que incluye (u, v) . Sea T cualquier árbol abarcador de costo mínimo para G . Agregar (u, v) a T debe formar un ciclo, ya que T es un árbol libre y, por tanto, satisface la propiedad (2) de los árboles libres. Este ciclo incluye la arista (u, v) . Así, debe haber otra arista (u', v') en T tal que $u' \in U$ y $v' \in V - U$, como se ilustra en la figura 7.5. Si no, no habría forma de que el ciclo fuera de u a v sin pasar por segunda vez por la arista (u, v) .

Al eliminar la arista (u', v') se rompe el ciclo y se obtiene un árbol abarcador T' cuyo costo en realidad no es mayor que el costo de T , ya que, por suposición, $c(u, v) \leq c(u', v')$. Así T' contradice la suposición de que no hay un árbol abarcador de costo mínimo que incluya (u, v) .

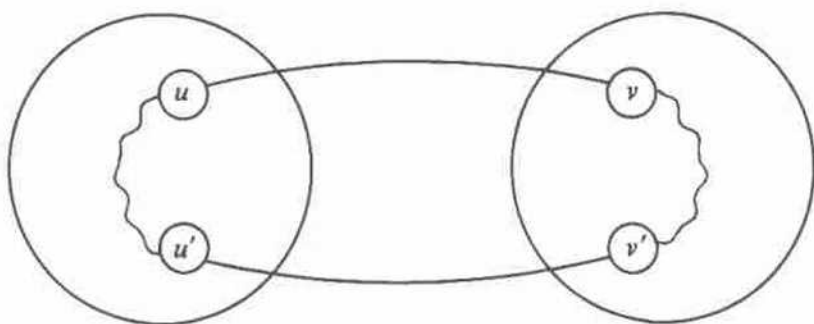


Fig. 7.5. Ciclo resultante.

Algoritmo de Prim

Existen dos técnicas populares que explotan la propiedad AAM para construir un árbol abarcador de costo mínimo a partir de un grafo ponderado $G = (V, A)$; una de ellas se conoce como algoritmo de Prim. Supóngase que $V = \{1, 2, \dots, n\}$. El algoritmo de Prim comienza cuando se asigna a un conjunto U un valor inicial $\{1\}$, en el cual «crece» un árbol abarcador, arista por arista. En cada paso localiza la arista más corta (u, v) que conecta U y $V - U$, y después agrega v , el vértice en $V - U$, a U . Este paso se repite hasta que $U = V$. El algoritmo se resume en la figura 7.6, y la secuencia de aristas agregadas a T para el grafo de la figura 7.4(a) se muestra en la figura 7.7.

```

procedure Prim (G: grafo; var T: conjunto de aristas);
  [Prim construye un árbol abarcador de costo mínimo T para G]
var
  U: conjunto de vértices;
  u, v: vértice;
begin
  T :=  $\emptyset$ ;
  U := {1};
  while U  $\neq$  V do begin
    sea (u, v) una arista de costo mínimo tal que u está en U y
      v en V - U;
    T := T  $\cup$  {(u, v)};
    U := U  $\cup$  {v};
  end
end; [Prim]

```

Fig. 7.6. Esbozo del algoritmo de Prim.

Una forma sencilla de encontrar la arista de menor costo entre *U* y *V* - *U* en cada paso es por medio de dos arreglos; uno, *MAS_CERCANO*[*i*], da el vértice en *U* que esté más cercano a *i* en *V* - *U*. El otro, *MENOR_COSTO*[*i*], da el costo de la arista (*i*, *MAS_CERCANO*[*i*]).

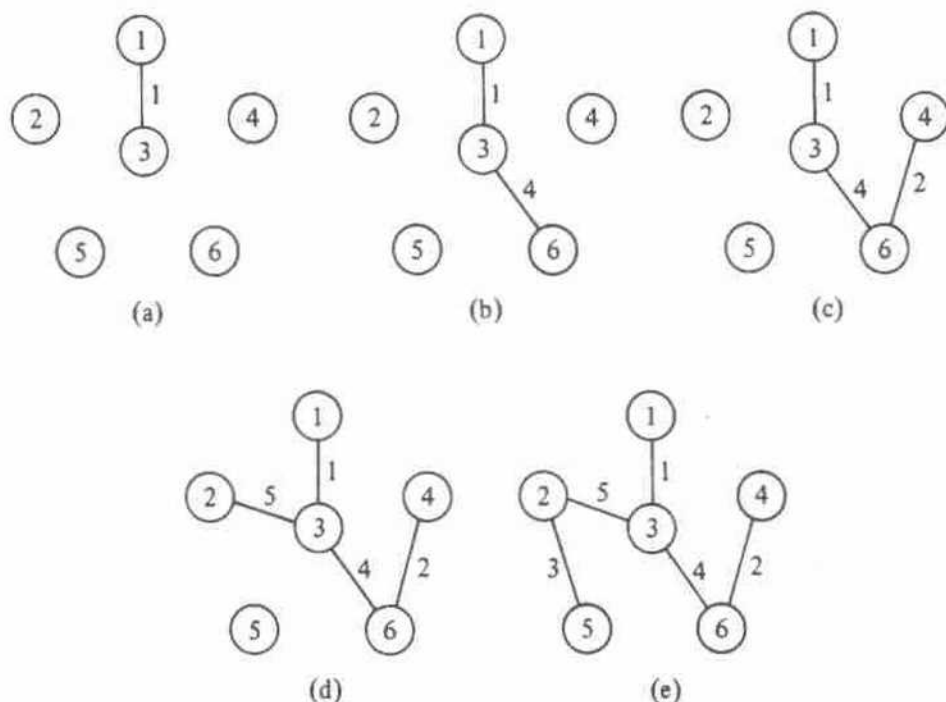


Fig. 7.7. Secuencias de aristas añadidas por el algoritmo de Prim.

En cada paso se revisa *MENOR_COSTO* para encontrar algún vértice, como k , en $V - U$ que esté más cercano a U . Se imprime la arista $(k, \text{MAS_CERCANO}[k])$. Entonces se actualizan los arreglos *MENOR_COSTO* y *MAS_CERCANO*, teniendo en cuenta el hecho de que k ha sido agregada a U . En la figura 7.8 se da una versión en Pascal de este algoritmo. Se supone que C es un arreglo de $n \times n$ tal que $C[i, j]$ es el costo de la arista (i, j) . Si la arista (i, j) no existe, se supone que $C[i, j]$ tiene un valor grande apropiado.

Si encuentra otro vértice k para el árbol abarcador, se hace que *MENOR_COSTO* $[k]$ sea *infinito*, un valor muy grande, de modo que este vértice ya no se considere en los recorridos siguientes para incluirlo en U . El valor *infinito* es mayor que el costo de cualquier arista o que el costo asociado a una arista no existente.

La complejidad de tiempo del algoritmo de Prim es $O(n^2)$, ya que se efectúan $n - 1$ iteraciones del ciclo de las líneas (4) a (16) y cada iteración del ciclo lleva un tiempo $O(n)$, debido a los ciclos más internos de las líneas (7) a (10) y (13) a (16). Conforme n crece, el rendimiento de este algoritmo puede dejar de ser satisfactorio. Ahora se presenta otro algoritmo, debido a Kruskal, para encontrar árboles abarcadores de costo mínimo cuyo rendimiento puede ser como máximo $O(a \log a)$, donde a es el número de aristas del grafo dado. Si a es mucho menor que n^2 , el algoritmo de Kruskal es superior, pero si es cercano a n^2 , se debe optar por el algoritmo de Prim.

procedure *Prim* (C : array[1.. n , 1.. n] of real);

| *Prim* imprime las aristas de un árbol abarcador de costo mínimo
para un grafo con vértices $\{1, 2, \dots, n\}$ y matriz de costo C
definida en las aristas |

var

MENOR_COSTO: array[1.. n] of real;

MAS_CERCANO: array[1.. n] of integer;

i, j, k, min : integer;

| i y j son índices. Durante una revisión del arreglo *MENOR_COSTO*,
 k es el índice del vértice más cercano encontrado hasta
ese punto, y $\text{min} = \text{MENOR_COSTO}[k]$ }

begin

- (1) **for** $i := 2$ to n **do begin**
- | asigna valor inicial al conjunto U sólo con el vértice 1 |
- (2) *MENOR_COSTO* $[i] := C[1, i]$;
- (3) *MAS_CERCANO* $[i] := 1$
- end;**
- (4) **for** $i := 2$ to n **do begin**
- | encuentra el vértice k fuera de U más cercano a algún vértice
 en U |
- (5) $\text{min} := \text{MENOR_COSTO}[2]$;
- (6) $k := 2$;
- (7) **for** $j := 3$ to n **do**
- (8) **if** *MENOR_COSTO* $[j] < \text{min}$ **then begin**

```

(9)           $min := MENOR\_COSTO[j];$ 
(10)          $k := j$ 
              end;
(11)         writeln(k, MAS_CERCANO[k]); { imprime la arista }
(12)          $MENOR\_COSTO[k] := infinito;$  { se añade k a U }
(13)         for j := 2 to n do { ajusta los costos de U }
(14)           if ( $C[k, j] < MENOR\_COSTO[j]$ ) and
              ( $MENOR\_COSTO[j] < infinito$ ) then begin
(15)              $MENOR\_COSTO[j] := C[k, j];$ 
(16)              $MAS\_CERCANO[j] := k$ 
              end
            end
          end
        end; { Prim }

```

Fig. 7.8. Algoritmo de Prim.

Algoritmo de Kruskal

Supóngase de nuevo que se tiene un grafo conexo $G = (V, A)$, con $V = \{1, 2, \dots, n\}$ y una función de costo c definida en las aristas de A . Otra forma de construir un árbol abarcador de costo mínimo para G es empezar con un grafo $T = (V, \emptyset)$ constituido sólo por los vértices de G y sin aristas. Por tanto, cada vértice es un componente conexo por sí mismo. Conforme el algoritmo avanza, habrá siempre una colección de componentes conexos, y para cada componente se seleccionarán las aristas que formen un árbol abarcador.

Para construir componentes cada vez mayores, se examinan las aristas a partir de A , en orden creciente de acuerdo con el costo. Si la arista conecta dos vértices que se encuentran en dos componentes conexos distintos, entonces se agrega la arista T . Se descartará la arista si conecta dos vértices contenidos en el mismo componente, ya que puede provocar un ciclo si se la añadiera al árbol abarcador para ese componente conexo. Cuando todos los vértices están en un solo componente, T es un árbol abarcador de costo mínimo para G .

Ejemplo 7.5. Considérese el grafo ponderado de la figura 7.4(a). La secuencia de aristas agregadas a T se muestra en la figura 7.9. Las aristas de costo 1, 2, 3 y 4 se consideran primero, y todas son aceptadas, ya que ninguna de ellas causa un ciclo. Las aristas (1, 4) y (3, 4) de costo 5 no pueden aceptarse, porque conectan vértices que están dentro del mismo componente en la figura 7.9(d) y, por tanto, pueden completar un ciclo. Sin embargo, la arista restante de costo 5, o sea (2, 3), no crea ciclos. Una vez que se acepta, el proceso termina. $\square$

Es posible aplicar este algoritmo mediante los conjuntos y sus operaciones asociadas de los capítulos 4 y 5. Primero se necesita un conjunto formado por las aristas de A . Al conjunto se le aplica en forma repetida el operador *SUPRIME-MIN* para seleccionar aristas en orden creciente de acuerdo con el costo. El conjunto de aristas, por tanto, forma una cola de prioridad, y entonces un árbol parcialmente ordenado es la estructura de datos más apropiada para usar aquí.

También se requiere mantener un conjunto de componentes conexos C . Las operaciones que se le aplican son:

1. **COMBINA**(A, B, C), para combinar los componentes A y B en C y llamar al resultado A o B en forma arbitraria $\dagger$.
2. **ENCUENTRA**(v, C), para devolver el nombre del componente de C , del cual el vértice v es miembro. Esta operación se usará para determinar si los dos vértices de una arista se encuentran en dos componentes distintos o en el mismo.
3. **INICIAL**(A, v, C), para que A sea el nombre de un componente que pertenece a C , y que inicialmente contiene sólo el vértice v .

Estas son las operaciones del TDA **COMBINA-ENCUENTRA** llamado **CONJUNTO-CE**, estudiado en la sección 5.5. En la figura 7.10 se muestra un esbozo de un programa llamado *Kruskal* para encontrar árboles abarcadores de costo mínimo con estas operaciones.

Se pueden emplear las técnicas desarrolladas en la sección 5.5 para implantar las operaciones utilizadas en este programa. El tiempo de ejecución de este programa depende de dos factores. Si hay a aristas, lleva un tiempo $O(a \log a)$ insertar las aristas en la cola de prioridad $\dagger\dagger$. En cada iteración del ciclo **while**, la obtención de la

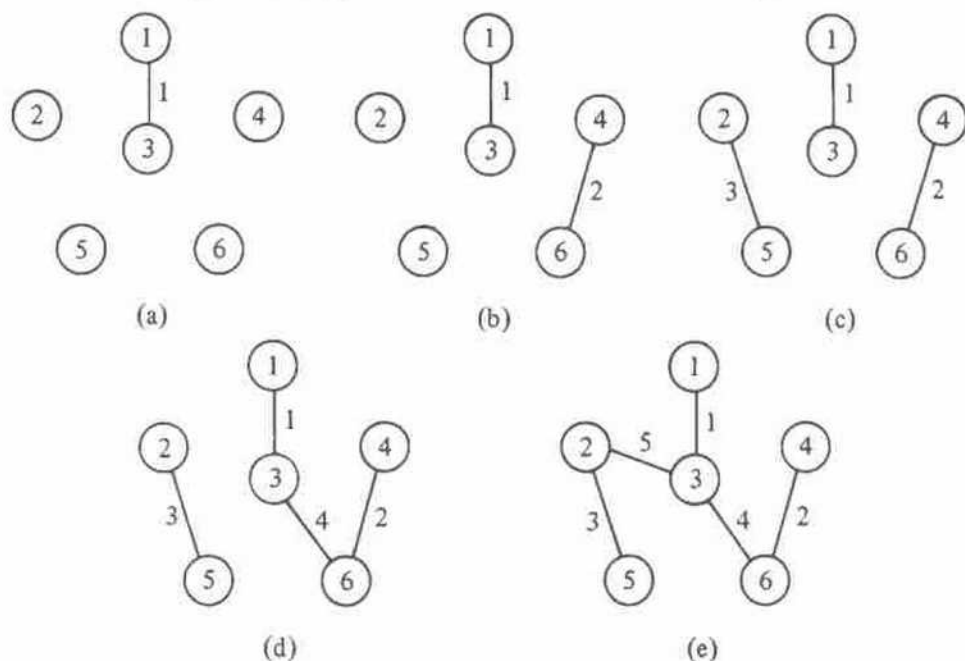


Fig. 7.9. Secuencia de aristas añadidas por el algoritmo de Kruskal.

$\dagger$ Obsérvese que **COMBINA** y **ENCUENTRA** son ligeramente distintas a las definiciones de la sección 5.5, ya que C es un parámetro que indica dónde se pueden encontrar A y B .

$\dagger\dagger$ Se puede asignar valor inicial a un árbol parcialmente ordenado de a elementos en un tiempo $O(a)$, si se hace todo de una vez. Esta técnica se analiza en la sección 8.4, aunque tal vez se debería utilizar aquí, ya que si se examinan menos de a aristas antes de encontrar el árbol abarcador de costo mínimo, se puede ahorrar un tiempo significativo.